

Variant calling pt 2

Michael Schatz

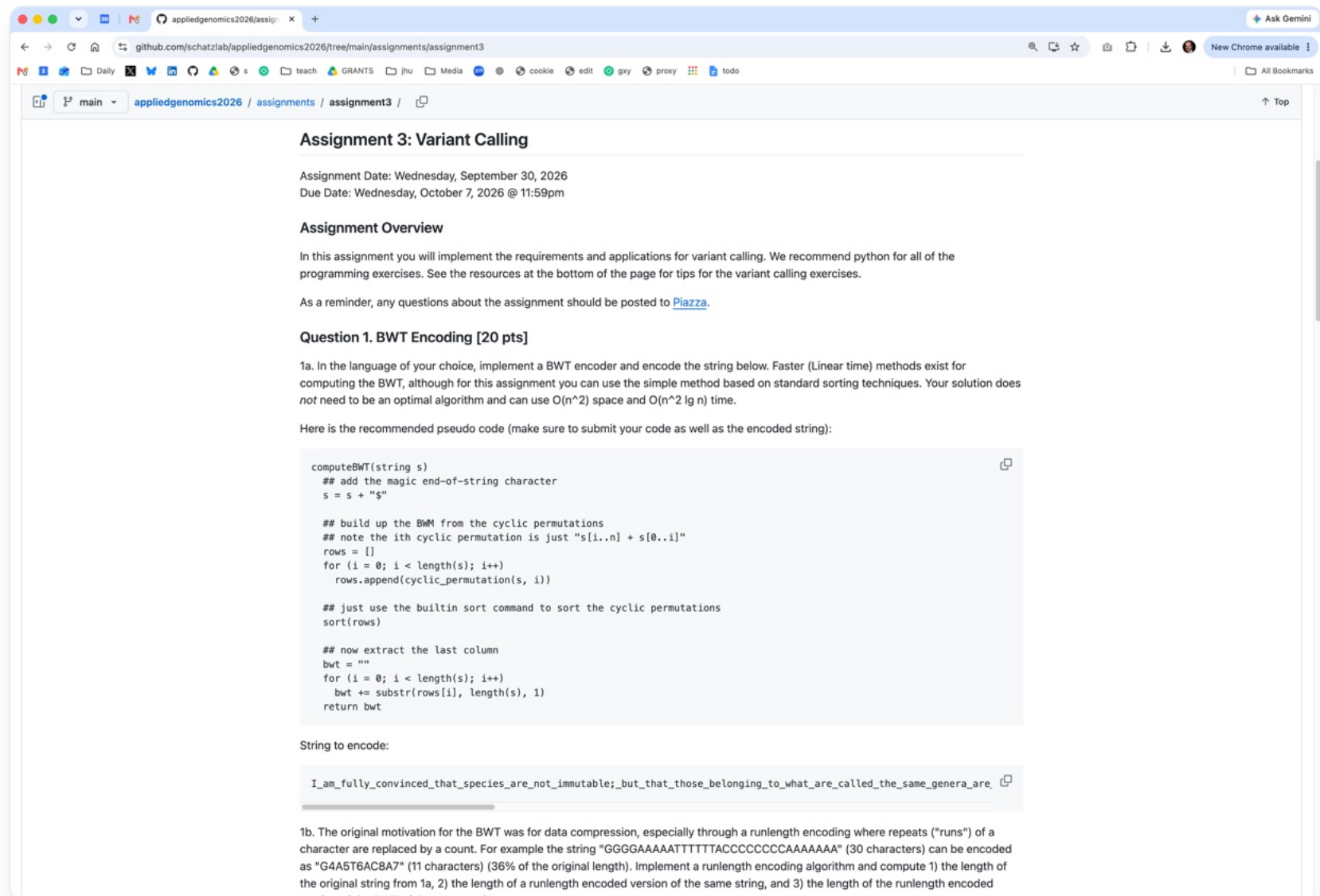
Sept 30, 2026

Lecture 9: Applied Comparative Genomics



Assignment 3!

Due Wednesday October 7 2026



The screenshot shows a web browser displaying a GitHub repository page for 'appliedgenomics2026/assignments/assignment3'. The page title is 'Assignment 3: Variant Calling'. It includes the assignment date (Wednesday, September 30, 2026) and the due date (Wednesday, October 7, 2026 @ 11:59pm). The 'Assignment Overview' section states that the assignment involves implementing requirements for variant calling, recommending Python. A reminder points to Piazza for questions. 'Question 1. BWT Encoding [20 pts]' is the first task, requiring a BWT encoder and encoding a specific string. Pseudo-code for the BWT encoding is provided, showing the process of adding a magic character, building cyclic permutations, sorting them, and extracting the last column. The string to be encoded is 'I_am_fully_convinced_that_species_are_not_immutable;but_that_those_belonging_to_what_are_called_the_same_genera_are'. The second part of the question (1b) discusses the original motivation for BWT for data compression using runlength encoding, with an example string and its encoded version.

Assignment 3: Variant Calling

Assignment Date: Wednesday, September 30, 2026
Due Date: Wednesday, October 7, 2026 @ 11:59pm

Assignment Overview

In this assignment you will implement the requirements and applications for variant calling. We recommend python for all of the programming exercises. See the resources at the bottom of the page for tips for the variant calling exercises.

As a reminder, any questions about the assignment should be posted to [Piazza](#).

Question 1. BWT Encoding [20 pts]

1a. In the language of your choice, implement a BWT encoder and encode the string below. Faster (Linear time) methods exist for computing the BWT, although for this assignment you can use the simple method based on standard sorting techniques. Your solution does not need to be an optimal algorithm and can use $O(n^2)$ space and $O(n^2 \lg n)$ time.

Here is the recommended pseudo code (make sure to submit your code as well as the encoded string):

```
computeBWT(string s)
    ## add the magic end-of-string character
    s = s + "$"

    ## build up the BWT from the cyclic permutations
    ## note the ith cyclic permutation is just "s[i..n] + s[0..i]"
    rows = []
    for (i = 0; i < length(s); i++)
        rows.append(cyclic_permutation(s, i))

    ## just use the builtin sort command to sort the cyclic permutations
    sort(rows)

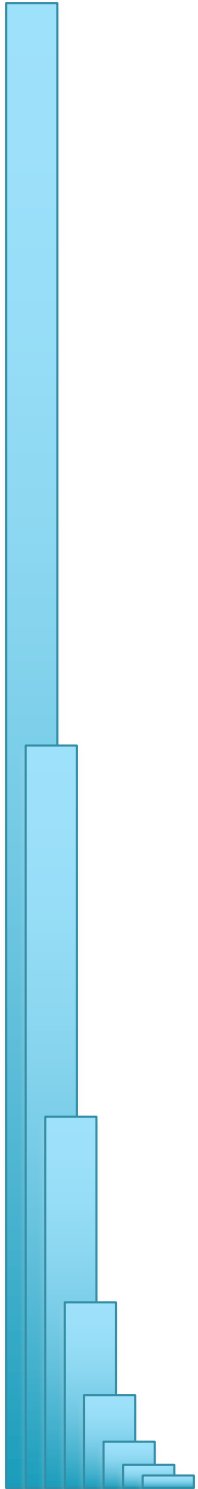
    ## now extract the last column
    bwt = ""
    for (i = 0; i < length(s); i++)
        bwt += substr(rows[i], length(s), 1)
    return bwt
```

String to encode:

```
I_am_fully_convinced_that_species_are_not_immutable;but_that_those_belonging_to_what_are_called_the_same_genera_are
```

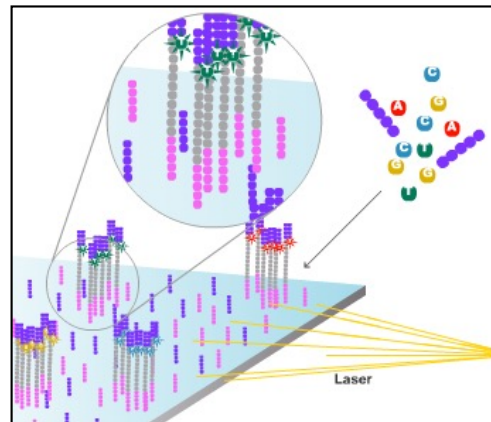
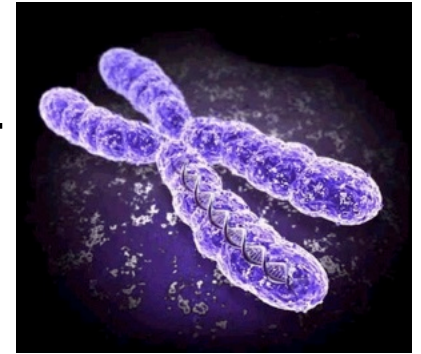
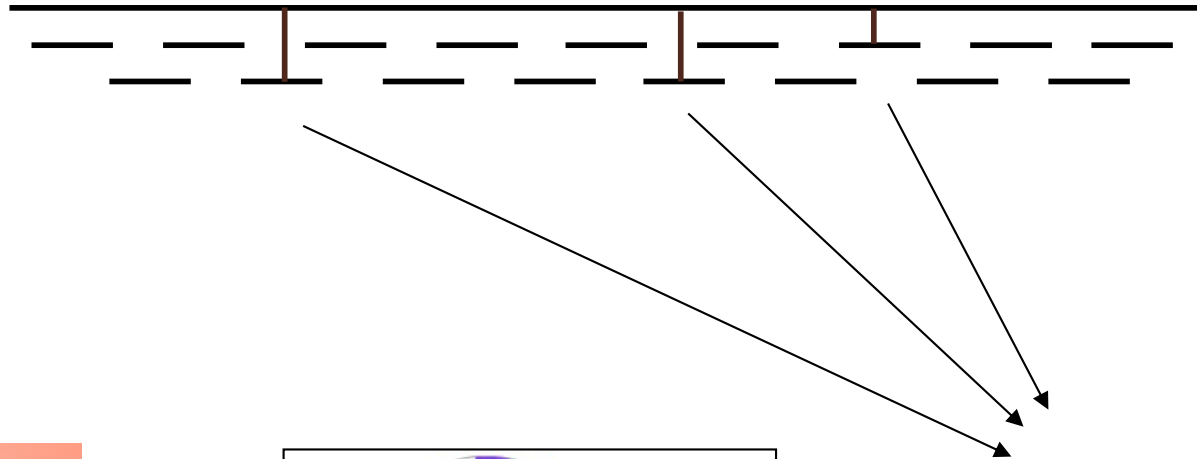
1b. The original motivation for the BWT was for data compression, especially through a runlength encoding where repeats ("runs") of a character are replaced by a count. For example the string "GGGGAAAAATTTTTACCCCCCAAAAAA" (30 characters) can be encoded as "G4A5T6ACBA7" (11 characters) (36% of the original length). Implement a runlength encoding algorithm and compute 1) the length of the original string from 1a, 2) the length of a runlength encoded version of the same string, and 3) the length of the runlength encoded version of the BWT of that same string.

Variant Calling Part 2



Personal Genomics

How does your genome compare to the reference?

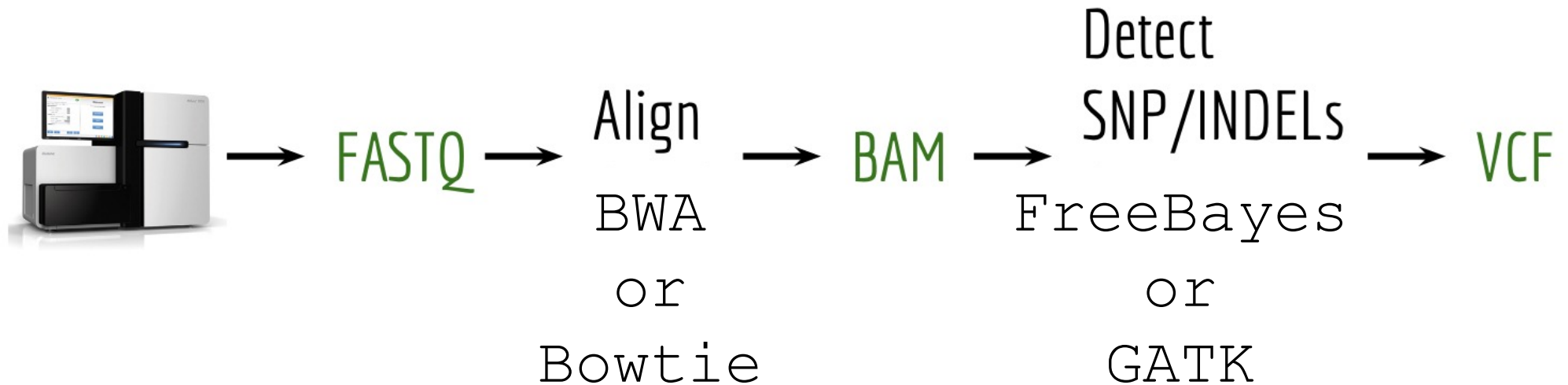


Heart Disease

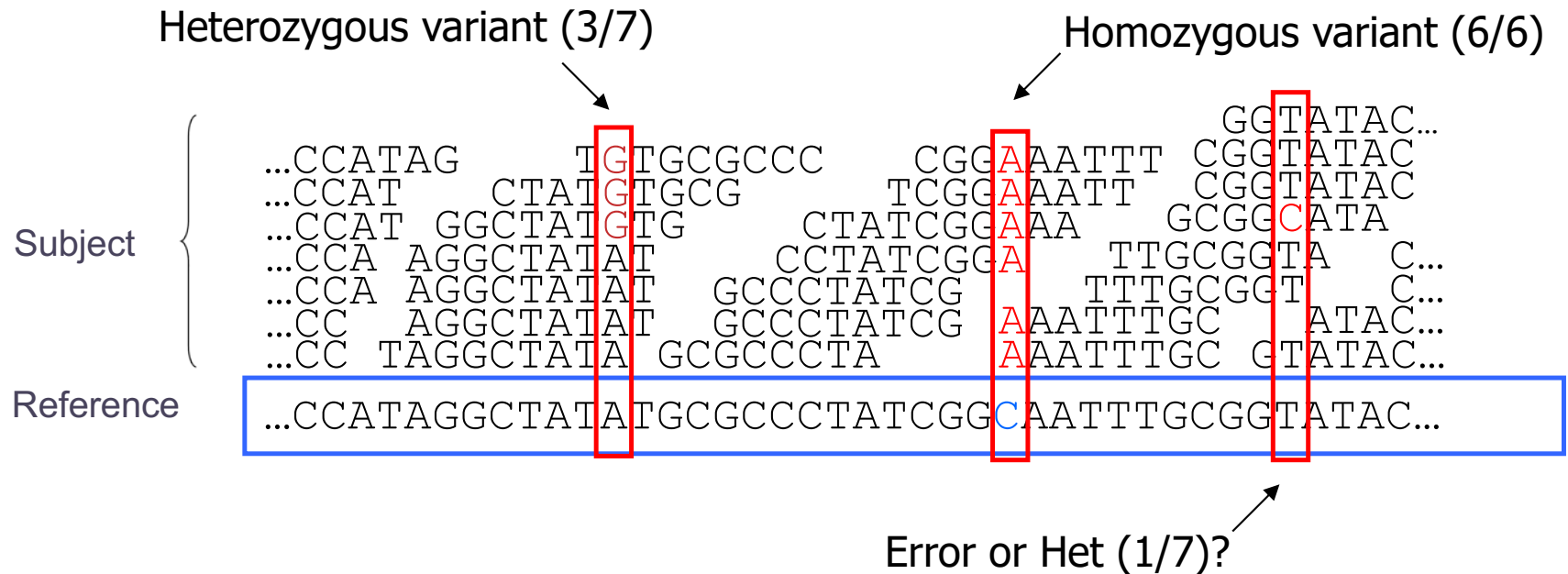
Cancer

Amazing actress

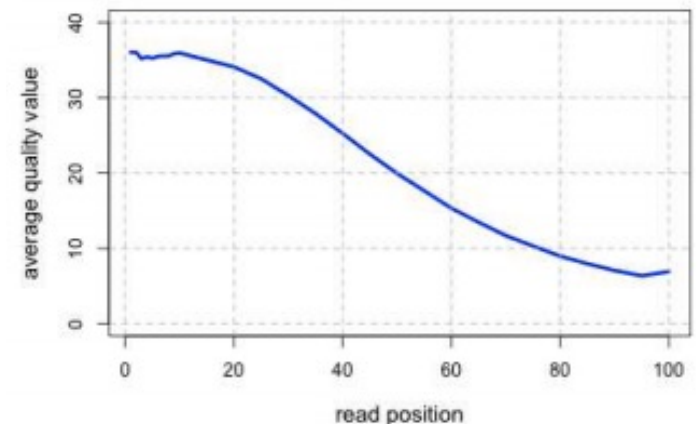
Variant Calling Overview



Genotyping Theory



- If there were no sequencing errors, identifying SNPs would be very easy: any time a read disagrees with the reference, it must be a variant!
- Sequencing instruments make mistakes
 - Quality of read decreases over the read length
- A single read differing from the reference is probably just an error, but it becomes more likely to be real as we see it multiple times



Brute Force Analysis

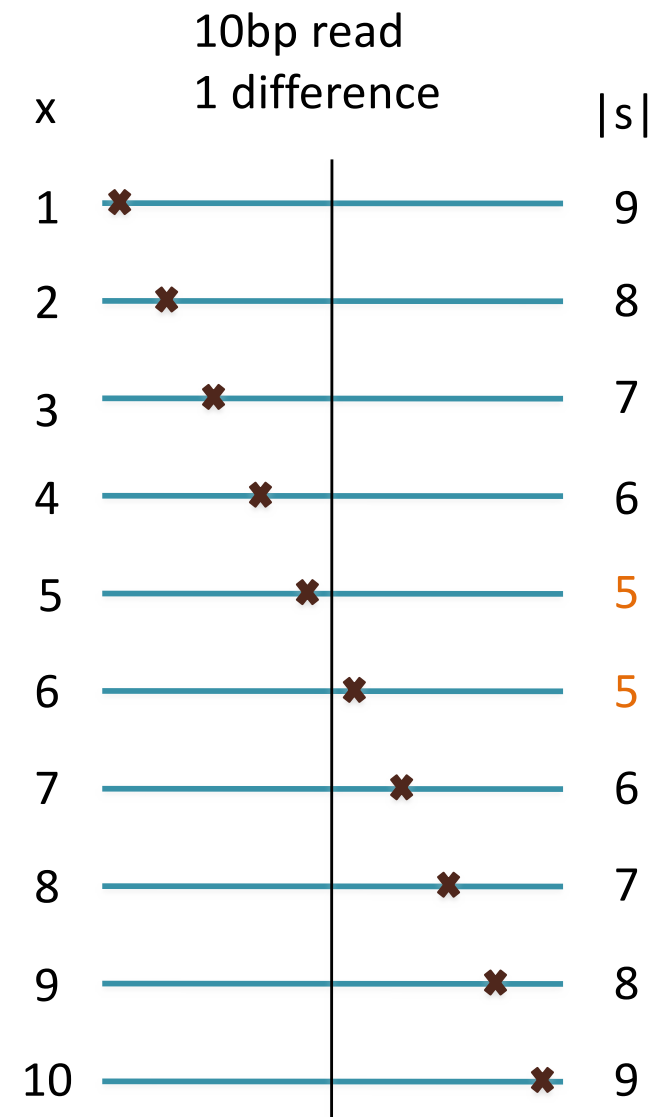


- Brute Force:
 - At every possible offset in the genome:
 - Do all of the characters of the query match?
- Analysis
 - Simple, easy to understand
 - Genome length = n [3B]
 - Query length = m [7]
 - Comparisons: $(n-m+1) * m$ [21B]
- Overall runtime: $O(nm)$
 - [How long would it take if we double the genome size, read length?]
 - [How long would it take if we double both?]

Seed-and-Extend Alignment

Theorem: An alignment of a sequence of length m with at most k differences **must** contain an exact match at least $s = m / (k + 1)$ bp long
(Baeza-Yates and Perleberg, 1996)

- Proof: Pigeonhole principle
 - 1 pigeon can't fill 2 holes
- Seed-and-extend search
 - Use an index to rapidly find short exact alignments to seed longer in-exact alignments
 - BLAST, MUMmer, Bowtie, BWA, SOAP, ...
 - Specificity of the depends on seed length
 - Guaranteed sensitivity for k differences
 - Also finds some (but not all) lower quality alignments <- heuristic



Binary Search Analysis

- Binary Search

Initialize search range to entire list

$mid = (hi+lo)/2$; $middle = suffix[mid]$

if query matches middle: done

else if query < middle: pick low range

else if query > middle: pick hi range

Repeat until done or empty range

[WHEN?]

- Analysis

- More complicated method

- How many times do we repeat?

- How many times can it cut the range in half?

- Find smallest x such that: $n/(2^x) \leq 1$; $x = \lg_2(n)$

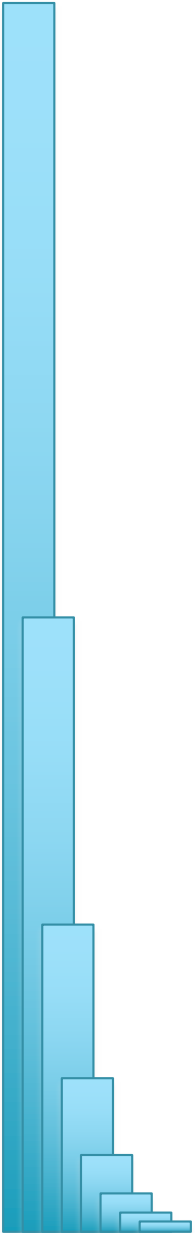
[32]

- Total Runtime: $O(m \lg n)$

- More complicated, but **much** faster!

- Looking up a query loops 32 times instead of 3B

Can be reduced to $O(m + \lg n)$
using an auxiliary data structure called the LCP array



Exact Matching Review & Overview

Where is GATTACA in the human genome?

Brute Force
(3 GB)

BANANA
BAN
ANA
NAN
ANA

$O(m * n)$

Slow & Easy

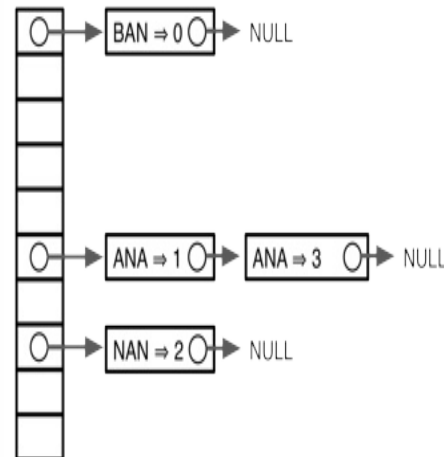
Suffix Array
(>15 GB)

6	\$
5	A\$
3	ANA\$
1	ANANA\$
0	BANANA\$
4	NA\$
2	NANA\$

$O(m + \lg n)$

Full-text index

Hash Table
(>15 GB)



$O(1)$

Fixed-length lookup

BWT
(3 GB)

BANANA\$
=>
\$BANANA**A**
A\$BANAN**N**
ANA\$BAN**N**
ANANA\$**B**
BANANA\$**S**
NA\$BANA**A**
NANA\$B**A**
=>
ANNB\$AA

$O(m)$

Full-text and concise

*** These are general techniques applicable to any text search problem ***

Algorithm Overview

1. Split read into segments

Read Read (reverse complement)

CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA TACAGGCCTGGGTAAAATAAGGCTGAGAGCTACTGG

Policy: extract 16 nt seed every 10 nt

Seeds

+, 0: CCAGTAGCTCTCAGCC	-, 0: TACAGGCCTGGGTAAA
+, 10: TCAGCCTTATTTTACC	-, 10: GGTAAAATAAGGCTGA
+, 20: TTTACCCAGGCCTGTA	-, 20: GGCTGAGAGCTACTGG

2. Lookup each segment and prioritize

The diagram illustrates the process of generating seed alignments from a set of seeds and an ungapped alignment with an FM Index.

Seeds:

- +, 0: CCAGTAGCTCTCAGCC
- +, 10: TCAGCCTTATTTTACC
- +, 20: TTTACCCAGGCCTGTA
- , 0: TACAGGCCCTGGGTAAA
- , 10: GGTA AAAATAAGGCTGA
- , 20: GGCTGAGAGCTACTGG

Ungapped alignment with FM Index:

```

      a a c
$ a c a a c g
a a c g $ a c
a c a a c g $
a c g $ a c a
[ c a a c a
[ c a a c a
g $ a c a a c
  
```

Seed alignments (as B ranges):

- { [211, 212], [212, 214] }
- { [653, 654], [651, 653] }
- { [684, 685] }
- { }
- { }
- { [624, 625] }

3. Evaluate end-to-end match

Extension candidates

SA:684, chr12:1955
SA:624, chr2:462
SA:211, chr4:762
SA:213, chr12:1935
SA:652, chr12:1945

→

SIMD dynamic programming aligner

→

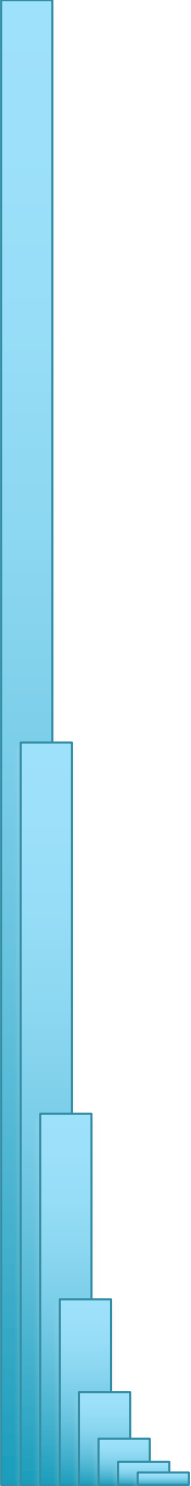
SAM alignments

```
r1 0 chr12 1936 0
36M * 0 0
CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA
IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
AS:i:0 XS:i:-2 XN:i:0
XM:i:0 XO:i:0 XG:i:0
NM:i:0 MD:Z:36 YT:Z:UU
YM:i:0
```

...

(Langmead & S)

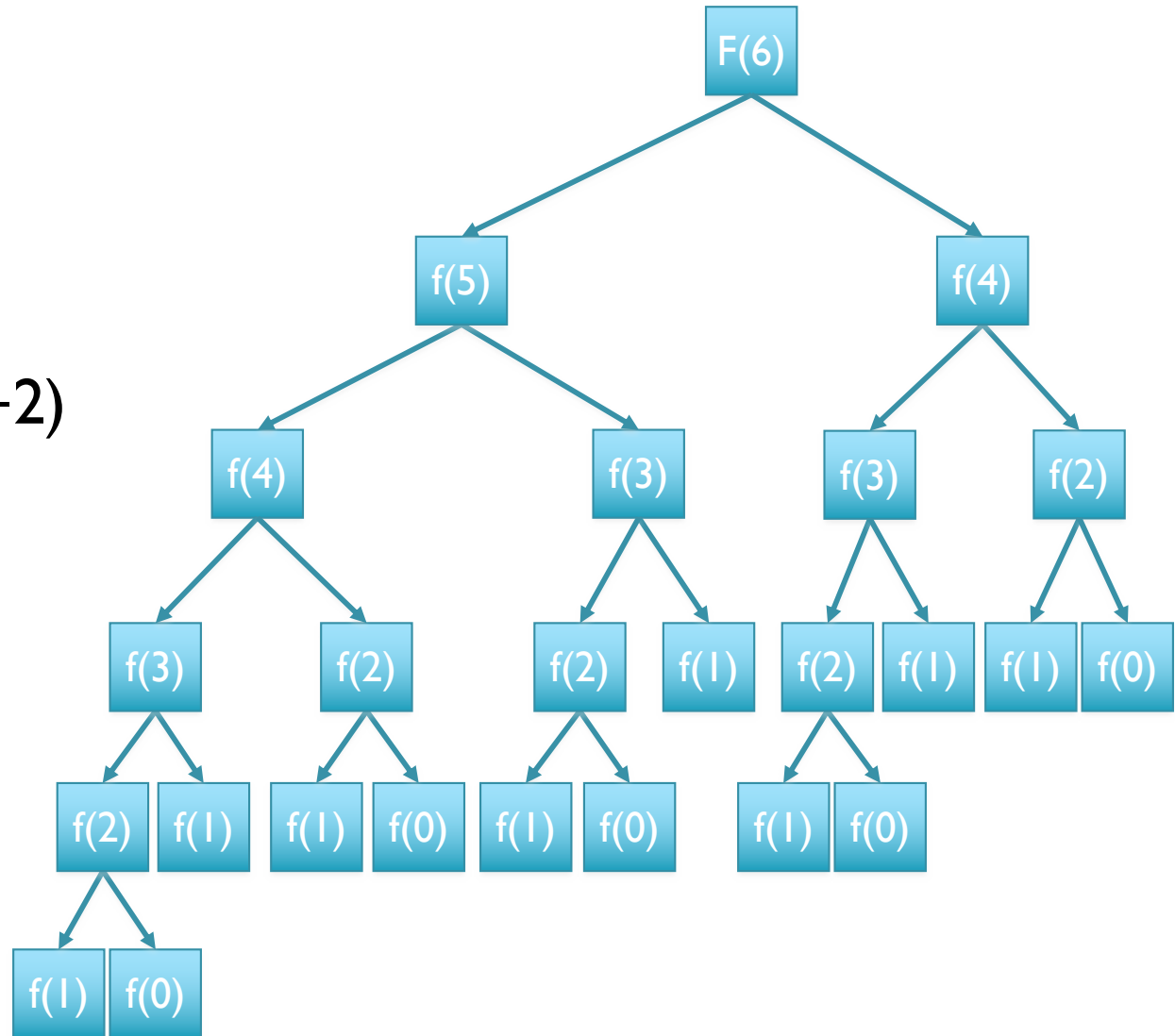
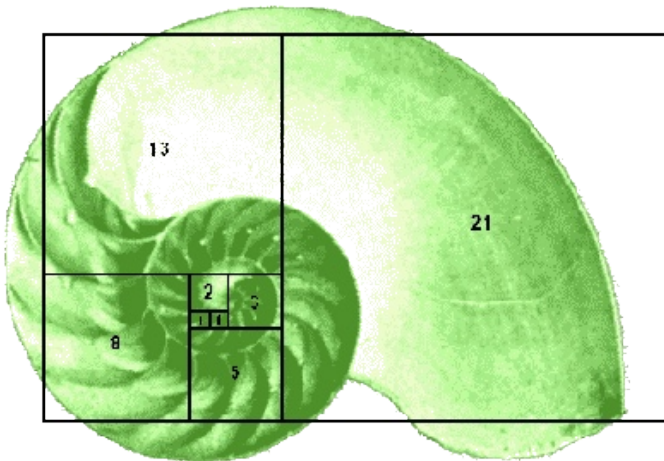
(Langmead & Salzberg, 2012)



Dynamic Programming and Edit Distance

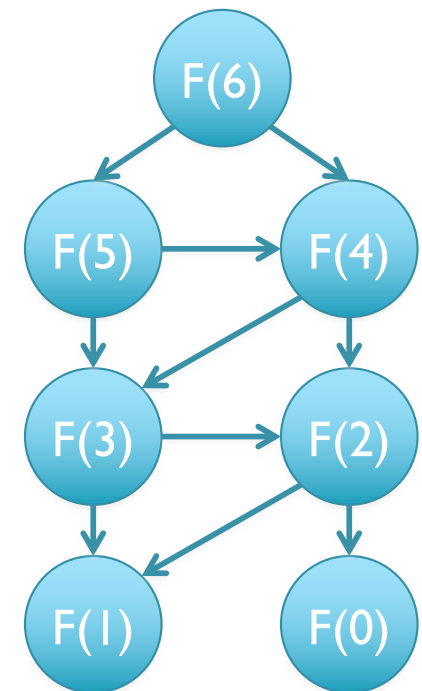
Fibonacci Sequence

```
def fib(n):  
    if n == 0 or n == 1:  
        return n  
    else:  
        return fib(n-1) + fib(n-2)
```



Dynamic Programming

- General approach for solving (some) complex problems
 - When applicable, the method takes far less time than naive methods.
 - Polynomial time ($O(n)$ or $O(n^2)$) instead of exponential time ($O(2^n)$ or $O(3^n)$)
- Requirements:
 - **Overlapping subproblems**
 - **Optimal substructure**
- Applications:
 - Fibonacci
 - Longest Increasing Subsequence (Bonus Slides!)
 - Sequence alignment, Dynamic Time Warp, Viterbi
- Not applicable:
 - Traveling salesman problem, Clique finding, Subgraph isomorphism, ...
 - The cheapest flight from airport A to airport B involves a single connection through airport C, but the cheapest flight from airport A to airport C involves a connection through some other airport D.

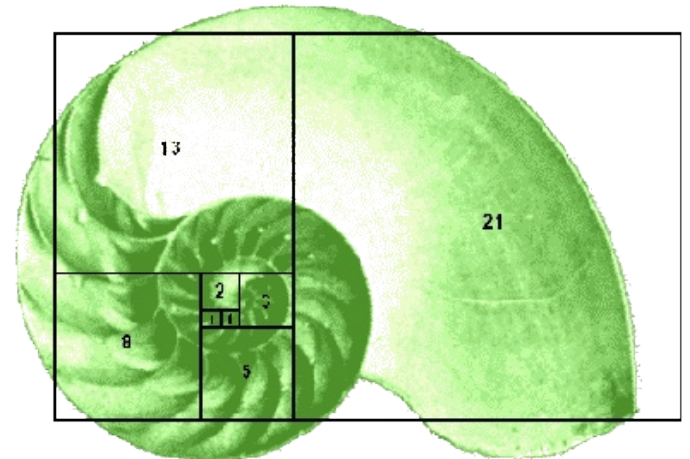


Bottom-up Fibonacci Sequence

```
def fib(n):  
    table = [0] * (n+1)  
    table[0] = 0  
    table[1] = 1  
    for i in range(2,n+1):  
        table[i] = table[i-2] + table[i-1]  
    return table[n]
```

0	1	2	3	4	5	6
0	1	1	2	3	5	8

[What is the running time?]



Similarity metrics

- Hamming distance

- Count the number of substitutions to transform one string into another

MIKESCHATZ

| | X | | XXXX |

MICESHATZZ

5

- Edit distance

- The minimum number of substitutions, insertions, or deletions to transform one string into another

MIKESCHAT-Z

| | X | | X | | | X |

MICES-HATZZ

3

Edit Distance Example

AGCACACA → ACACACTA in 4 steps

AGCACACA → (1. change G to C)

ACCACACA → (2. delete C)

ACACACA → (3. change A to T)

ACACACT → (4. insert A after T)

ACACACTA → done

[Is this the best we can do?]

Edit Distance Example

AGCACACA → ACACACTA in 3 steps

AGCACACA → (1. change G to C)

ACCACACA → (2. delete C)

ACACACA → (3. insert T after 3rd C)

ACACACTA → done

[Is this the best we can do?]

Reverse Engineering Edit Distance

$$D(\text{AGCACACA}, \text{ACACACTA}) = ?$$

Imagine we already have the optimal alignment of the strings, the last column can only be 1 of 3 options:

...M	...I	...D
...A	...-	...A
...A	...A	...-

The optimal alignment of last two columns is then 1 of 9 possibilities

...MM	...IM	...DM	...MI	...II	...DI	...MD	...ID	...DD
...CA	...-A	...CA	...A-	...--	...A-	...CA	...-A	...CA
...TA	...TA	...-A	...TA	...TA	...-A	...A-	...A-	...--

The optimal alignment of the last three columns is then 1 of 27 possibilities...

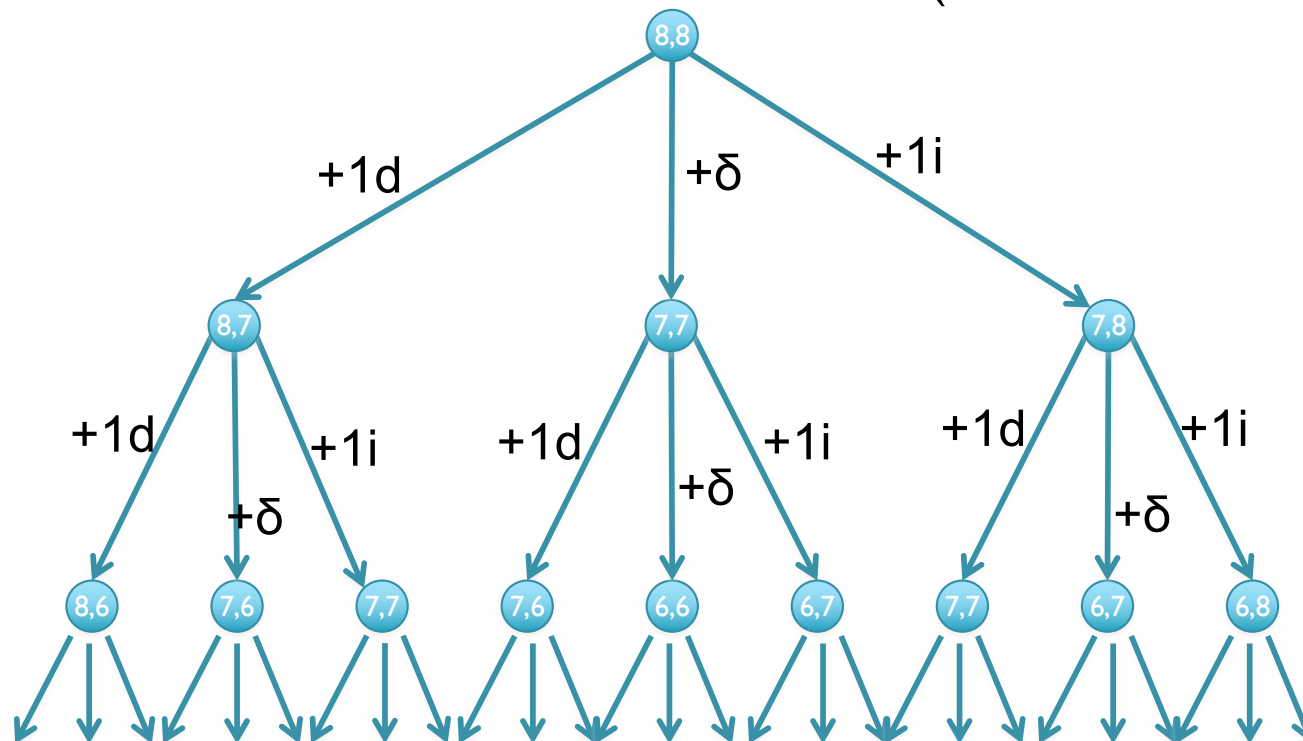
...M...	...I...	...D...
...X...	...-...	...X...
...Y...	...Y...	...-...

Eventually spell out every possible sequence of {I,M,D}

Recursive solution

- Computation of D is a recursive process.
 - At each step, we only allow matches, substitutions, and indels
 - $D(i,j)$ in terms of $D(i',j')$ for $i' \leq i$ and $j' \leq j$.

$$D(\text{AGCACACA}, \text{ACACACTA}) = \min \{ D(\text{AGCACACA}, \text{ACACACT}) + 1, \\ D(\text{AGCACAC}, \text{ACACACTA}) + 1, \\ D(\text{AGCACAC}, \text{ACACACT}) + \delta(\text{A}, \text{A}) \}$$



[What is the running time?]

Dynamic Programming

- We could code this as a recursive function call...
...with an exponential number of function evaluations
- There are only $(n+1) \times (m+1)$ pairs i and j
 - We are evaluating $D(i,j)$ multiple times
- Compute $D(i,j)$ bottom up.
 - Start with smallest $(i,j) = (1,1)$.
 - Store the intermediate results in a table.
 - Compute $D(i,j)$ *after* $D(i-1,j)$, $D(i,j-1)$, and $D(i-1,j-1)$

Recurrence Relation for D

Find the edit distance (minimum number of operations to convert one string into another) in $O(mn)$ time

- Base conditions:

- $D(i,0) = i$, for all $i = 0, \dots, n$
- $D(0,j) = j$, for all $j = 0, \dots, m$

- For $i > 0, j > 0$:

$$D(i,j) = \min \left\{ \begin{array}{ll} D(i-1,j) + 1, & // \text{align 0 chars from S, 1 from T} \\ D(i,j-1) + 1, & // \text{align 1 char from S, 0 from T} \\ D(i-1,j-1) + \delta(S(i),T(j)) & // \text{align 1+1 chars} \end{array} \right\}$$

[Why do we want the min?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1								
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

[What does the initialization mean?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1	0							
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,A] = \min\{D[A,]+1, D[,A]+1, D[,]+ \delta(A,A)\}$$

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1	0	1						
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,AC] = \min\{D[A,A]+1, D[,AC]+1, D[,A]+\delta(A,C)\}$$

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1	0	1	2					
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,ACA] = \min\{D[A,AC]+1, D[,ACA]+1, D[,AC]+\delta(A,A)\}$$

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>
A	1	0	1	2	3	4	5	6	<u>7</u>
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A, ACACACTA] = 7$$

-----A

*****|

ACACACTA

[What about the other A?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	5	6	7	8
A	1	0	1	2	3	<u>4</u>	5	6	7
G	2	1	1	2	3	4	<u>5</u>	<u>6</u>	<u>7</u>
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[AG, ACACACTA] = 7$$

-----AG--

***** | ***

ACACACTA

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	<u>0</u>	1	2	3	4	5	6	7	8
A	1	<u>0</u>	1	2	3	4	5	6	7
G	2	<u>1</u>	1	2	3	4	5	6	7
C	3	2	<u>1</u>	2	2	3	4	5	6
A	4	3	2	<u>1</u>	2	2	3	4	5
C	5	4	3	2	<u>1</u>	2	2	3	4
A	6	5	4	3	2	<u>1</u>	2	3	3
C	7	6	5	4	3	2	<u>1</u>	<u>2</u>	3
A	8	7	6	5	4	3	2	2	<u>2</u>

$$D[\text{AGCACACA}, \text{ACACACTA}] = 2$$

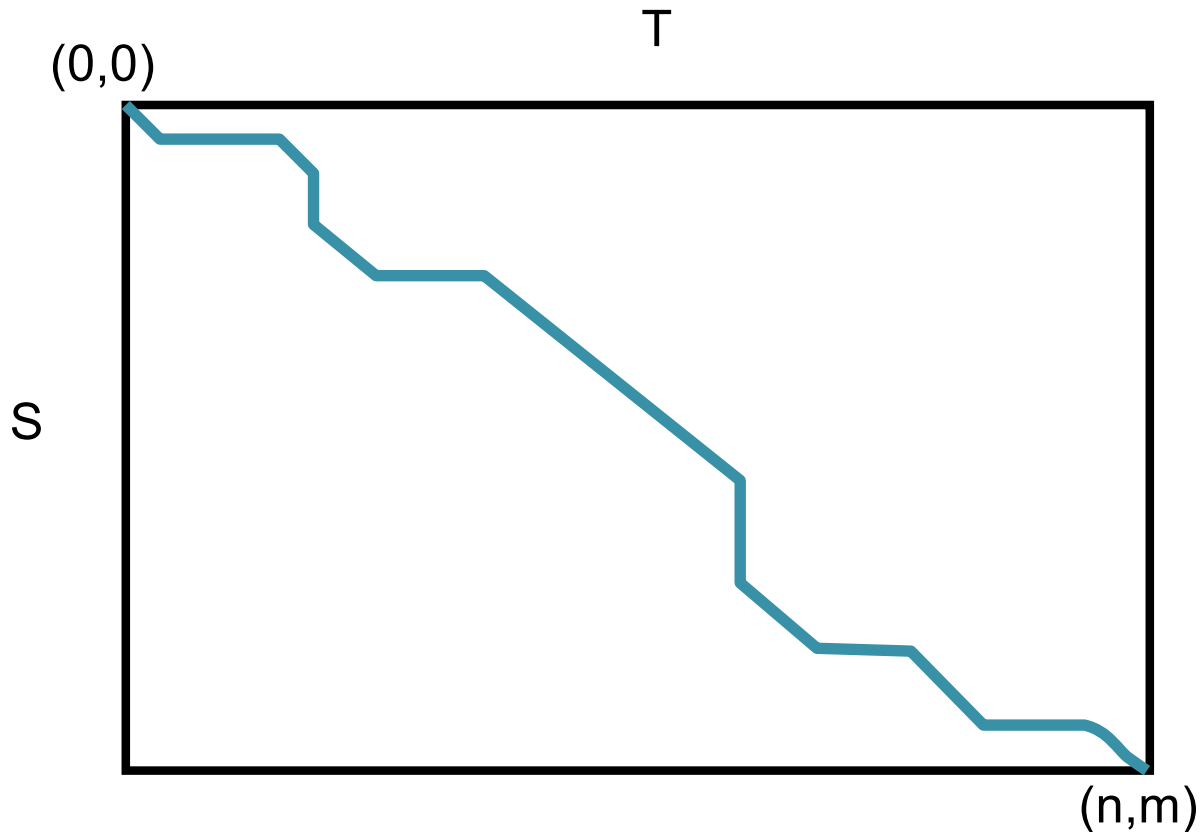
AGCACAC-A

|*| | | | |*|

A-CACACTA

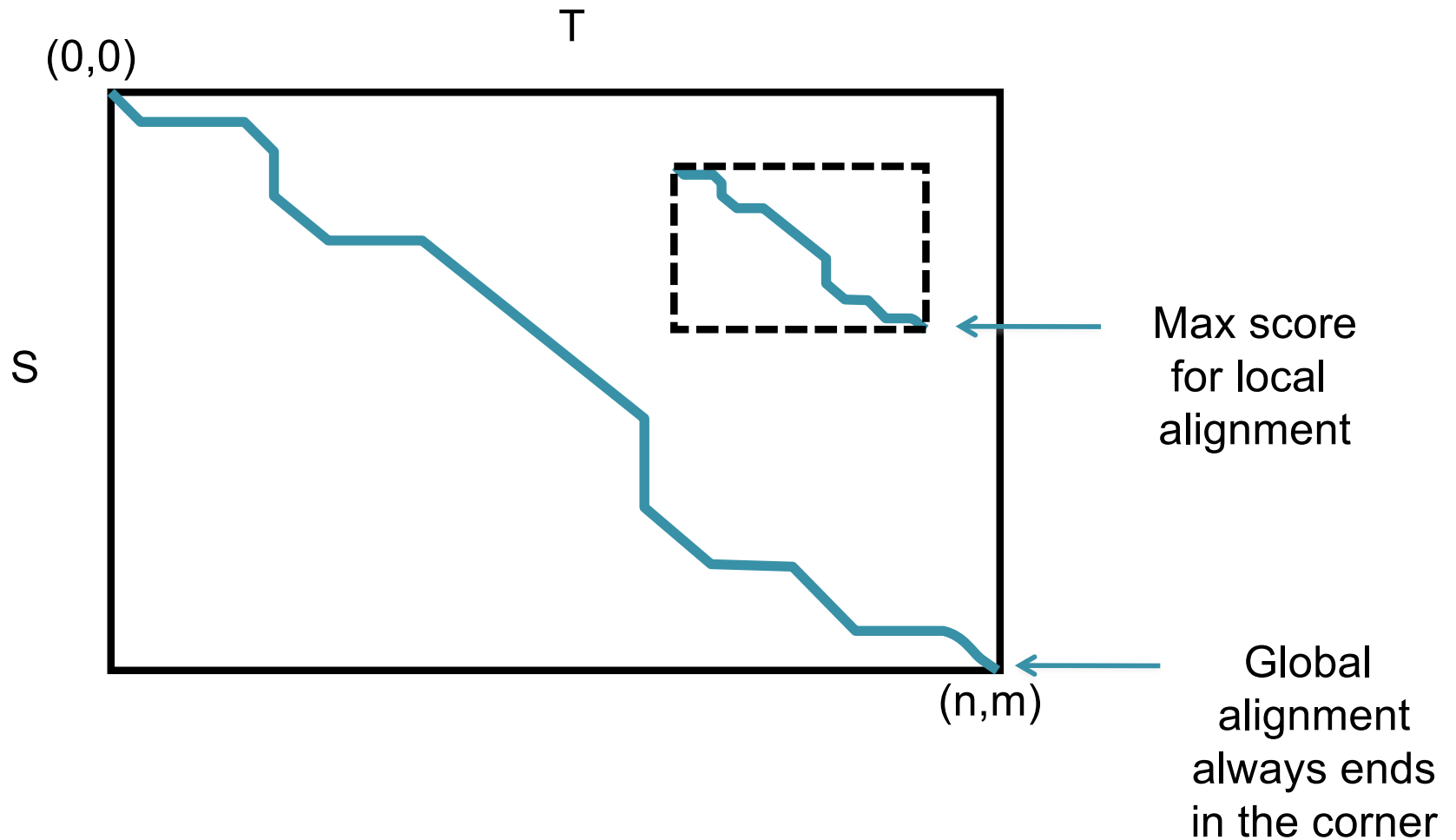
[Can we do it any better?]

Global Alignment Schematic



- A high quality alignment will stay close to the diagonal
 - If we are only interested in high quality alignments, we can skip filling in cells that can't possibly lead to a high quality alignment
 - Find the global alignment with at most edit distance d : $O(2dn)$

Global vs Local Alignment Schematic



SAM / BAM Files

```
Coord      12345678901234 5678901234567890123456789012345
ref         AGCATGTTAGATAA**GATAGCTGTGCTAGTAGGCAGTCAGCGCCAT

+r001/1      TTAGATAAAGGATA*CTG
+r002        aaaAGATAA*GGATA
+r003        gcctaAGCTAA
+r004                ATAGCT.....TCAGC
-r003                ttagctTAGGC
-r001/2                        CAGCGGCAT
```

```
@HD VN:1.6 SO:coordinate
@SQ SN:ref LN:45
r001 99 ref 7 30 8M2I4M1D3M = 37 39 TTAGATAAAGGATACTG *
r002 0 ref 9 30 3S6M1P1I4M * 0 0 AAAAGATAAGGATA *
r003 0 ref 9 30 5S6M * 0 0 GCCTAAGCTAA * SA:Z:ref,29,-,6H5M,17,0;
r004 0 ref 16 30 6M14N5M * 0 0 ATAGCTTCAGC *
r003 2064 ref 29 17 6H5M * 0 0 TAGGC * SA:Z:ref,9,+,5S6M,30,1;
r001 147 ref 37 30 9M = 7 -39 CAGCGGCAT * NM:i:1
```

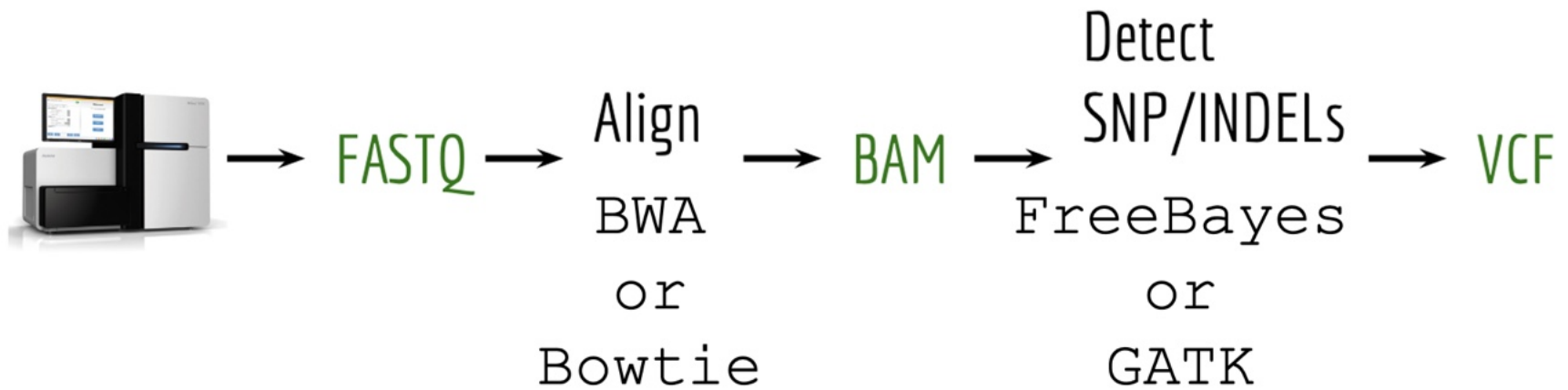
SAM / BAM Files

1.4 The alignment section: mandatory fields

In the SAM format, each alignment line typically represents the linear alignment of a segment. Each line consists of 11 or more TAB-separated fields. The first eleven fields are always present and in the order shown below; if the information represented by any of these fields is unavailable, that field's value will be a placeholder, either '0' or '*' as determined by the field's type. The following table gives an overview of these mandatory fields in the SAM format:

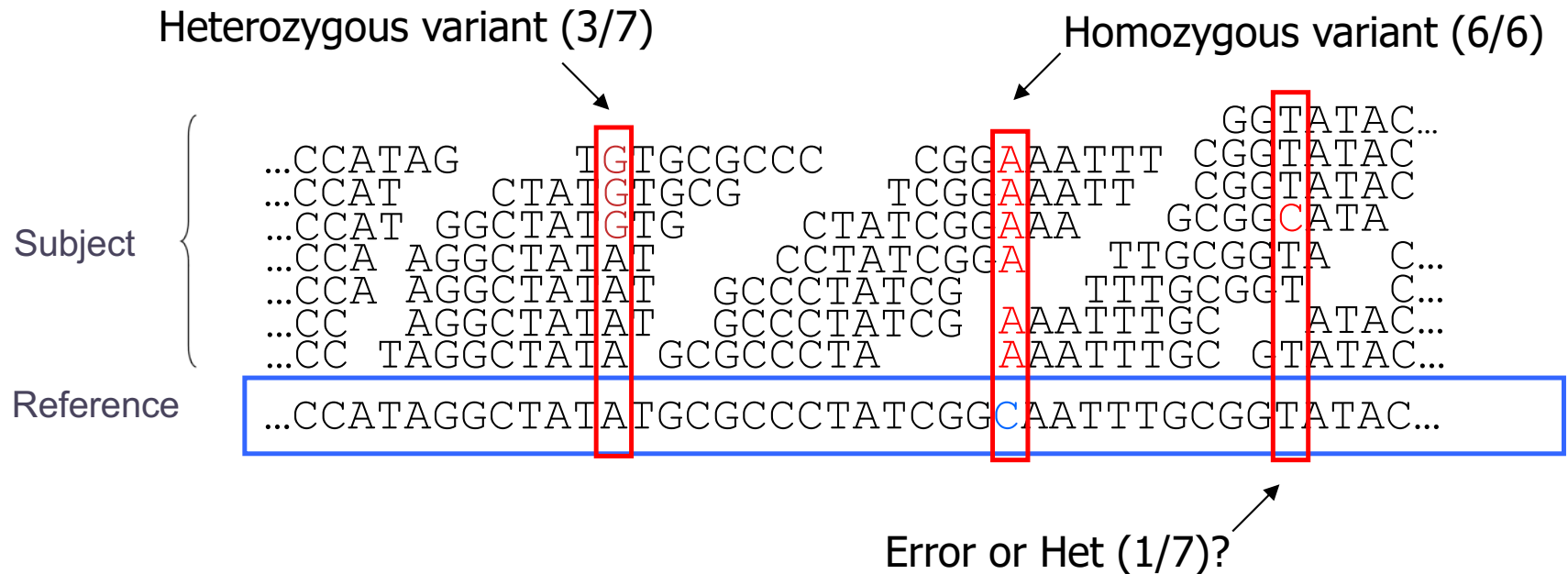
Col	Field	Type	Regexp/Range	Brief description
1	QNAME	String	[!-?A-~]{1,254}	Query template NAME
2	FLAG	Int	[0, 2 ¹⁶ - 1]	bitwise FLAG
3	RNAME	String	* [:rname:^*=] [:rname:]*	Reference sequence NAME ¹¹
4	POS	Int	[0, 2 ³¹ - 1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0, 2 ⁸ - 1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [:rname:^*=] [:rname:]*	Reference name of the mate/next read
8	PNEXT	Int	[0, 2 ³¹ - 1]	Position of the mate/next read
9	TLEN	Int	[-2 ³¹ + 1, 2 ³¹ - 1]	observed Template LENgth
10	SEQ	String	* [A-Za-z=.]+	segment SEquence
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

¹¹Reference sequence names may contain any printable ASCII characters with the exception of certain punctuation characters, and may not start with '*' or '='. See Section 1.2.1 for details and an explanation of the [:rname:] notation.

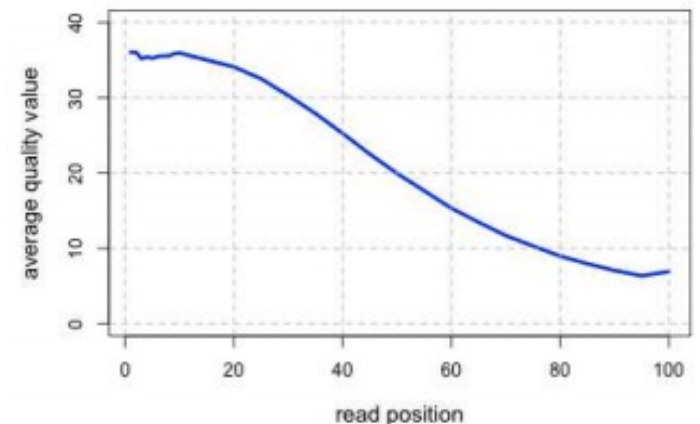


Part 2: Variant Calling

Genotyping Theory



- If there were no sequencing errors, identifying SNPs would be very easy: any time a read disagrees with the reference, it must be a variant!
- Sequencing instruments make mistakes
 - Quality of read decreases over the read length
- A single read differing from the reference is probably just an error, but it becomes more likely to be real as we see it multiple times



The Binomial Distribution: Adventures in Coin Flipping

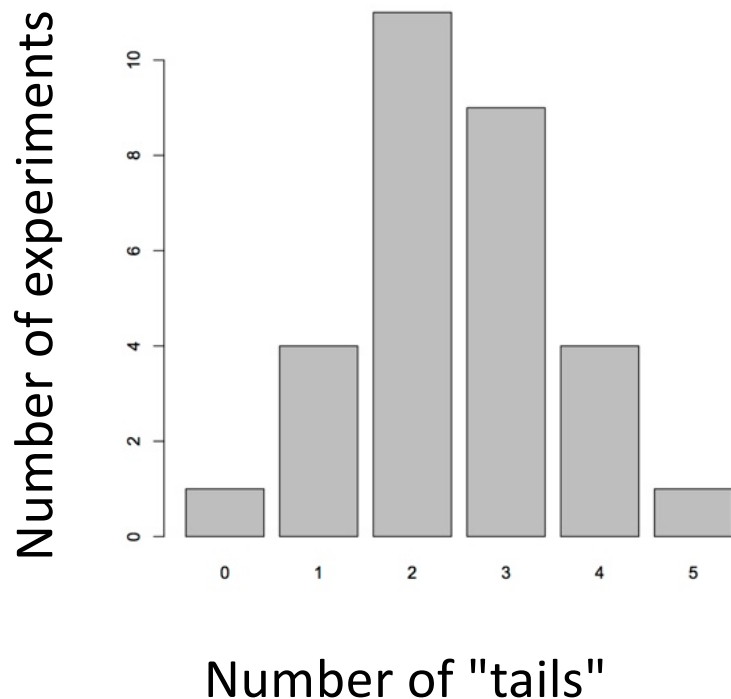


$P(\text{heads}) = 0.5$



$P(\text{tails}) = 0.5$

What is the distribution of tails
(alternate alleles) do we expect to see
after 5 tosses (sequence reads)?



R code:

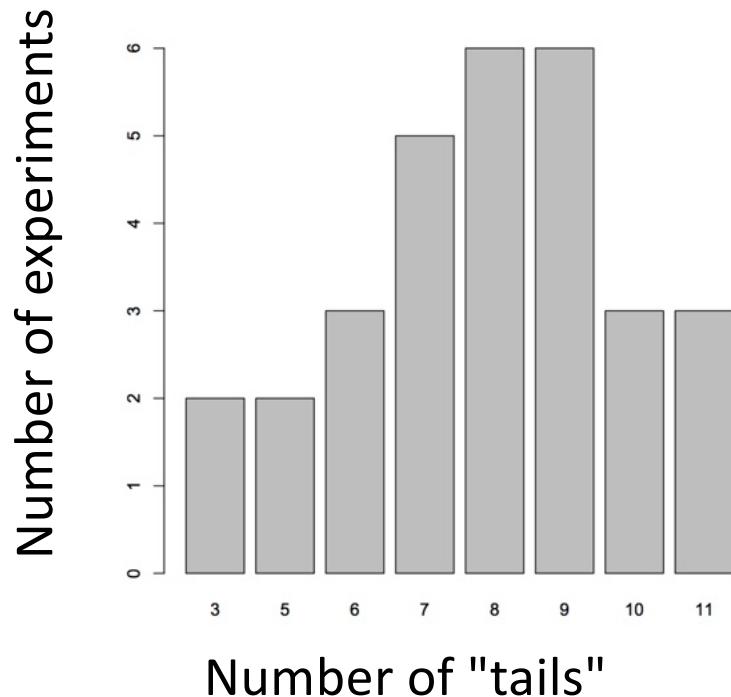
```
barplot(table(rbinom(30, 5, 0.5)))
```

30 experiments (students tossing coins)

5 tosses each

Probability of Tails

What is the distribution of tails
(alternate alleles) do we expect to see
after 15 tosses (sequence reads)?



R code:

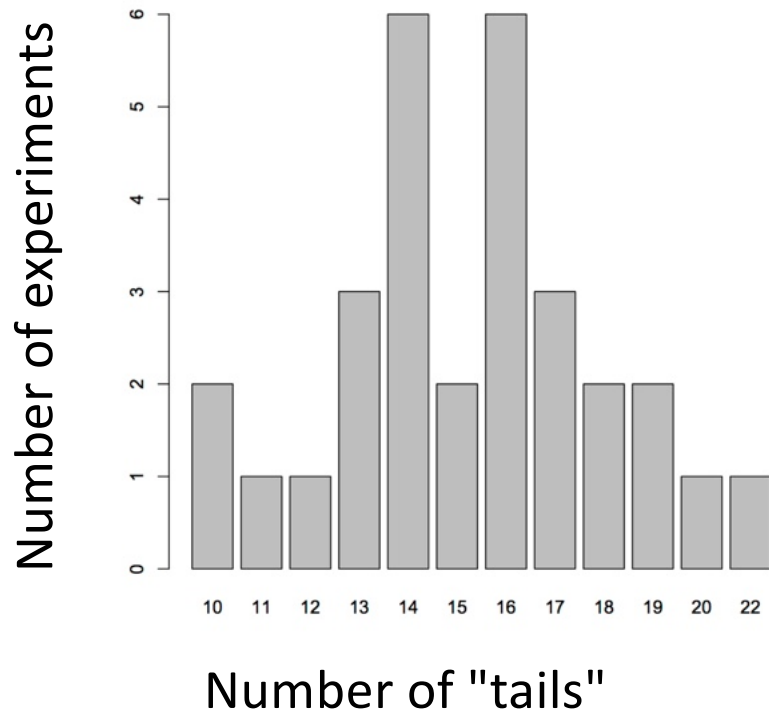
```
barplot(table(rbinom(30, 15, 0.5)))
```

30 experiments (students tossing coins)

15 tosses each

Probability of Tails

What is the distribution of tails
(alternate alleles) do we expect to see
after 30 tosses (sequence reads)?



R code:

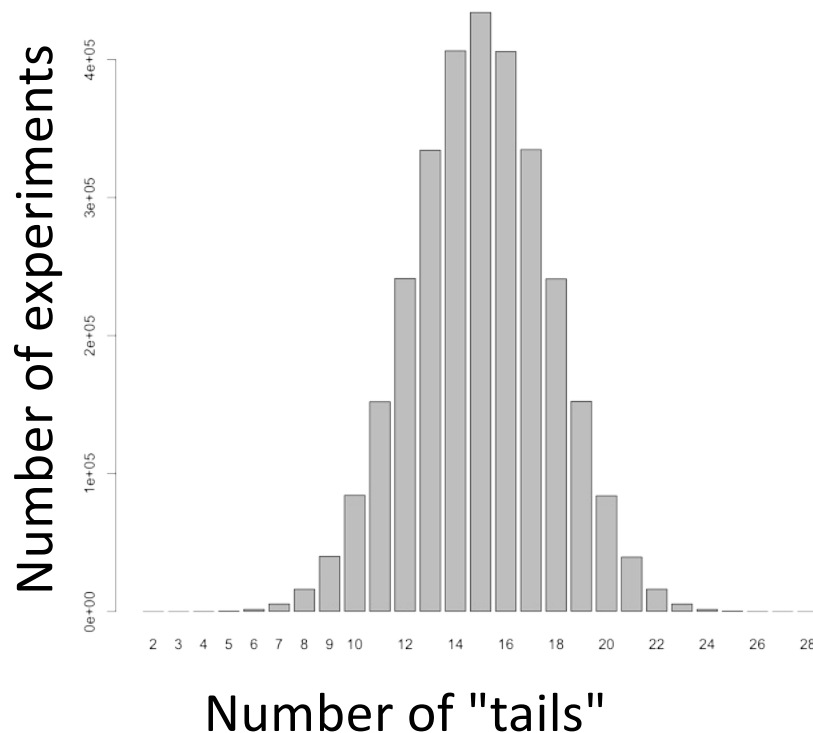
```
barplot(table(rbinom(30, 30, 0.5)))
```

30 experiments (students tossing coins)

30 tosses each

Probability of Tails

What is the distribution of tails
(alternate alleles) do we expect to see
after 30 tosses (sequence reads)?



R code:

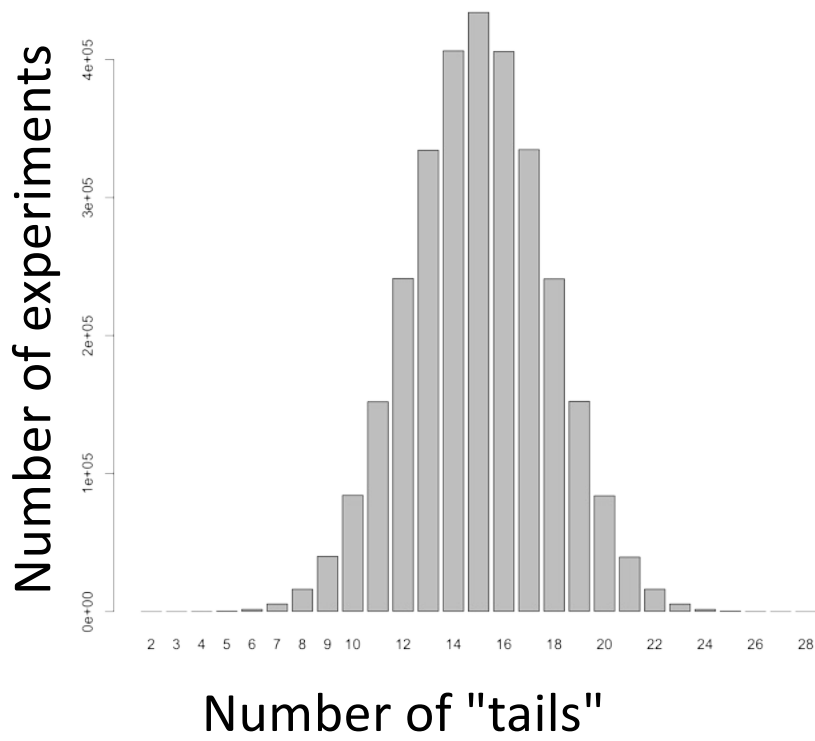
```
barplot(table(rbinom(3e6, 30, 0.5)))
```

3M experiments (students tossing coins)

30 tosses each

Probability of Tails

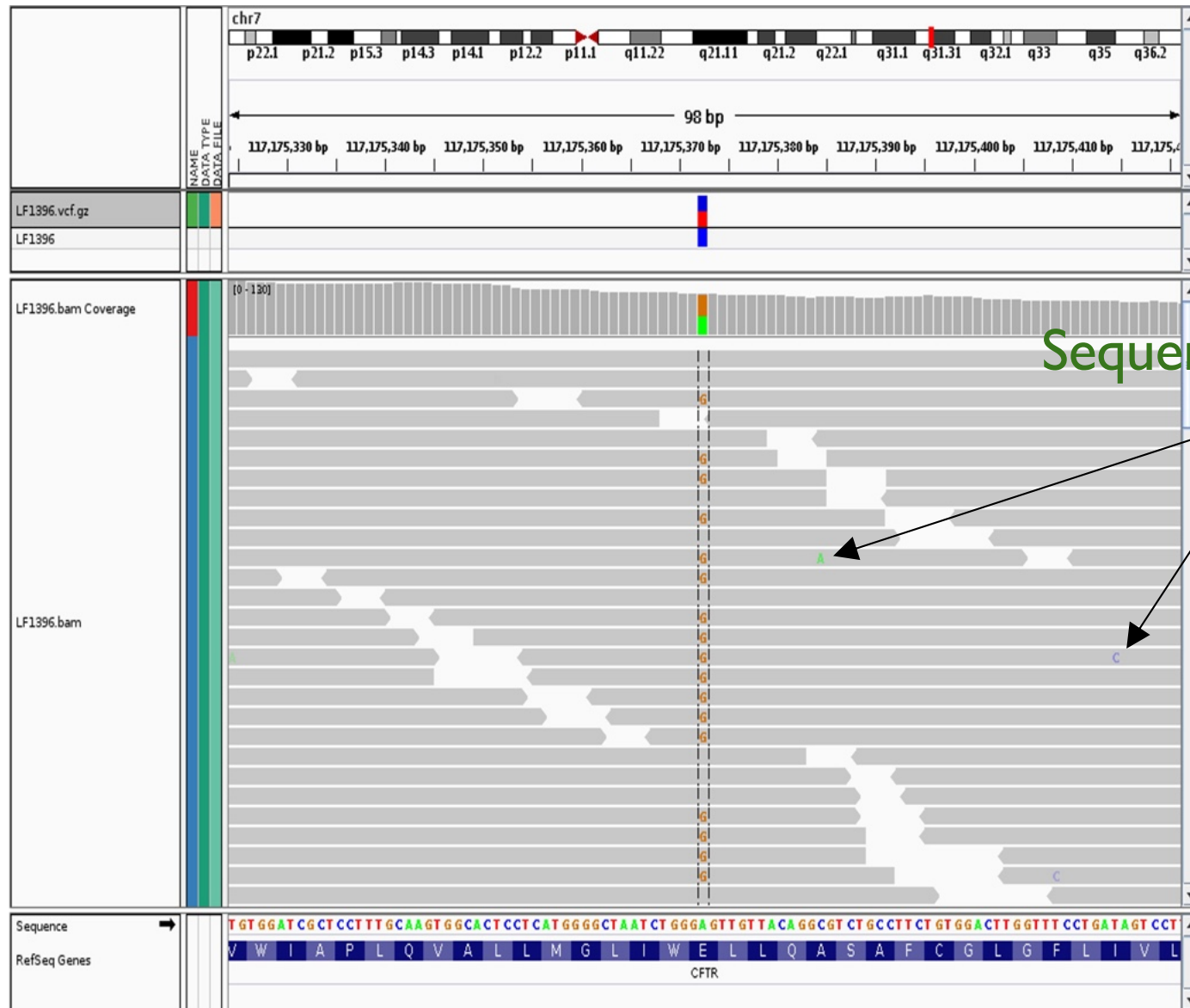
So, with 30 tosses (reads), we are much more likely to see an even mix of alternate and reference alleles at a heterozygous locus in a genome



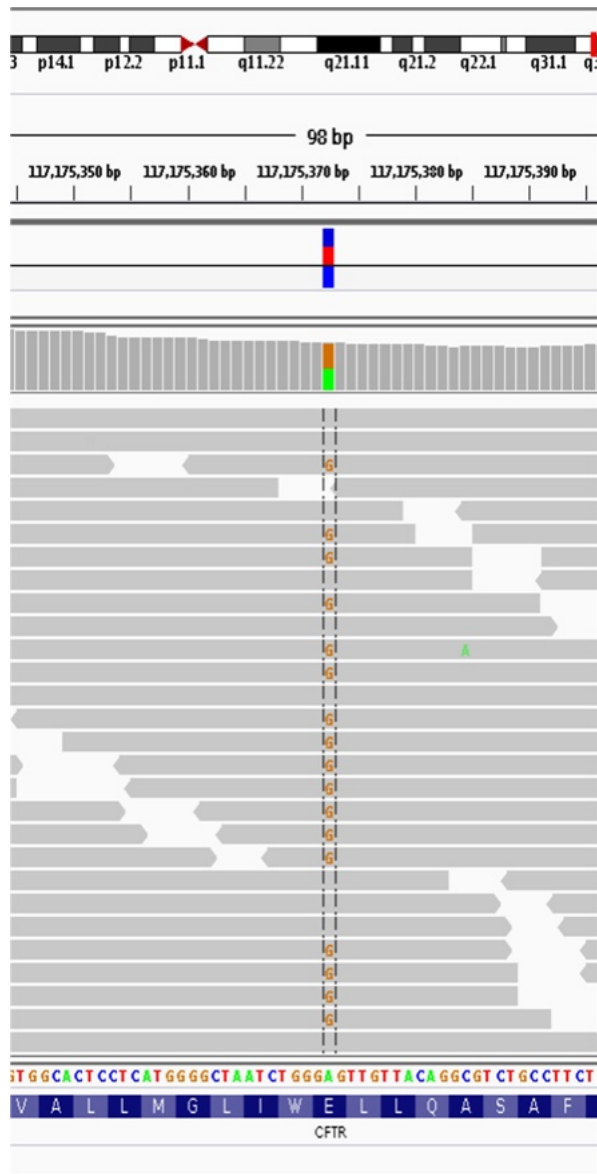
This is why at least a "30X" (30 fold sequence coverage) genome is recommended: it confers sufficient power to distinguish heterozygous alleles and from mere sequencing errors

$$P(3/30 \text{ het}) <?> P(3/30 \text{ err})$$

Sequencing errors fall out as noise (most of the time)

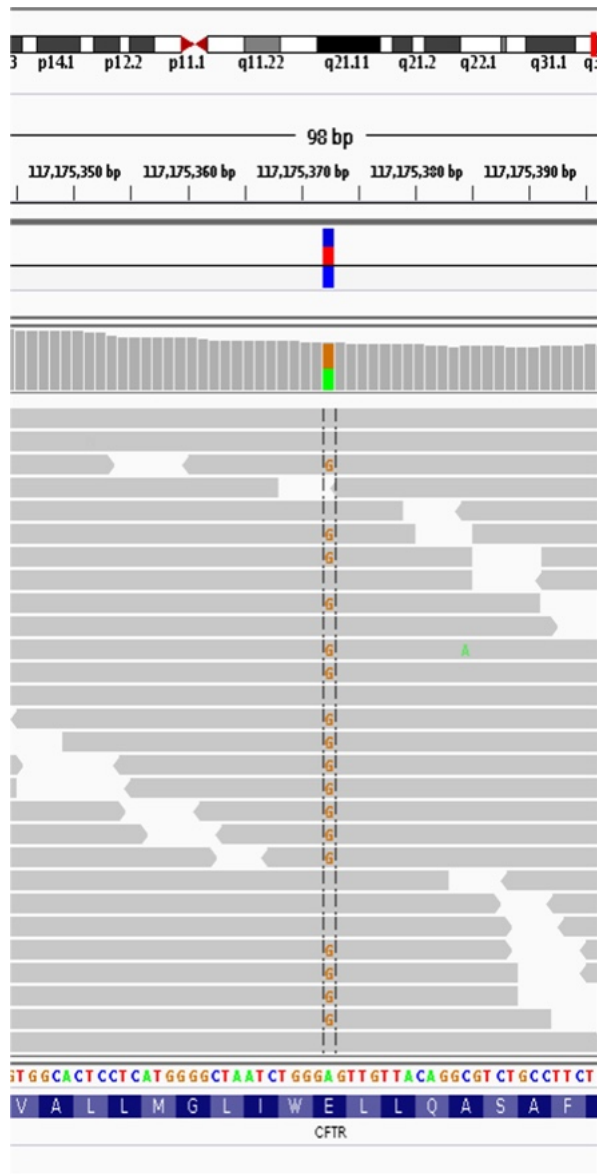


What information is needed to decide if a variant exists?



- Depth of coverage at the locus
- Bases observed at the locus
- The base qualities of each allele
- The strand composition
- Mapping qualities
- Proper pairs?
- Expected polymorphism rate

Bayesian SNP calling



$$P(\text{SNP} | \text{Data}) = \frac{P(\text{Data} | \text{SNP}) * P(\text{SNP})}{P(\text{Data})}$$

PolyBayes: The first statistically rigorous variant detection tool.

letter

© 1999 Nature America Inc. • <http://genetics.nature.com>

A general approach to single-nucleotide polymorphism discovery

Gabor T. Marth¹, Ian Korf¹, Mark D. Yandell¹, Raymond T. Yeh¹, Zhijie Gu², Hamideh Zakeri², Nathan O. Stitzel¹, LaDeana Hillier¹, Pui-Yan Kwok² & Warren R. Gish¹

Bayesian
posterior
probability

$$P(SNP) = \sum_{all \ variable \ S} \frac{\frac{P(S_1 | R_1) \cdot \dots \cdot P(S_N | R_N)}{P_{Prior}(S_1) \cdot \dots \cdot P_{Prior}(S_N)} \cdot P_{Prior}(S_1, \dots, S_N)}{\sum_{S_{i_1} \in [A,C,G,T]} \dots \sum_{S_{i_N} \in [A,C,G,T]} \frac{P(S_{i_1} | R_1)}{P_{Prior}(S_{i_1})} \cdot \dots \cdot \frac{P(S_{i_N} | R_1)}{P_{Prior}(S_{i_N})} \cdot P_{Prior}(S_{i_1}, \dots, S_{i_N})}$$

Base call + Base quality

Expected (prior) polymorphism rate

Probability of observed base composition (should model sequencing error rate)

PolyBayes: The first statistically rigorous variant detection tool.

letter

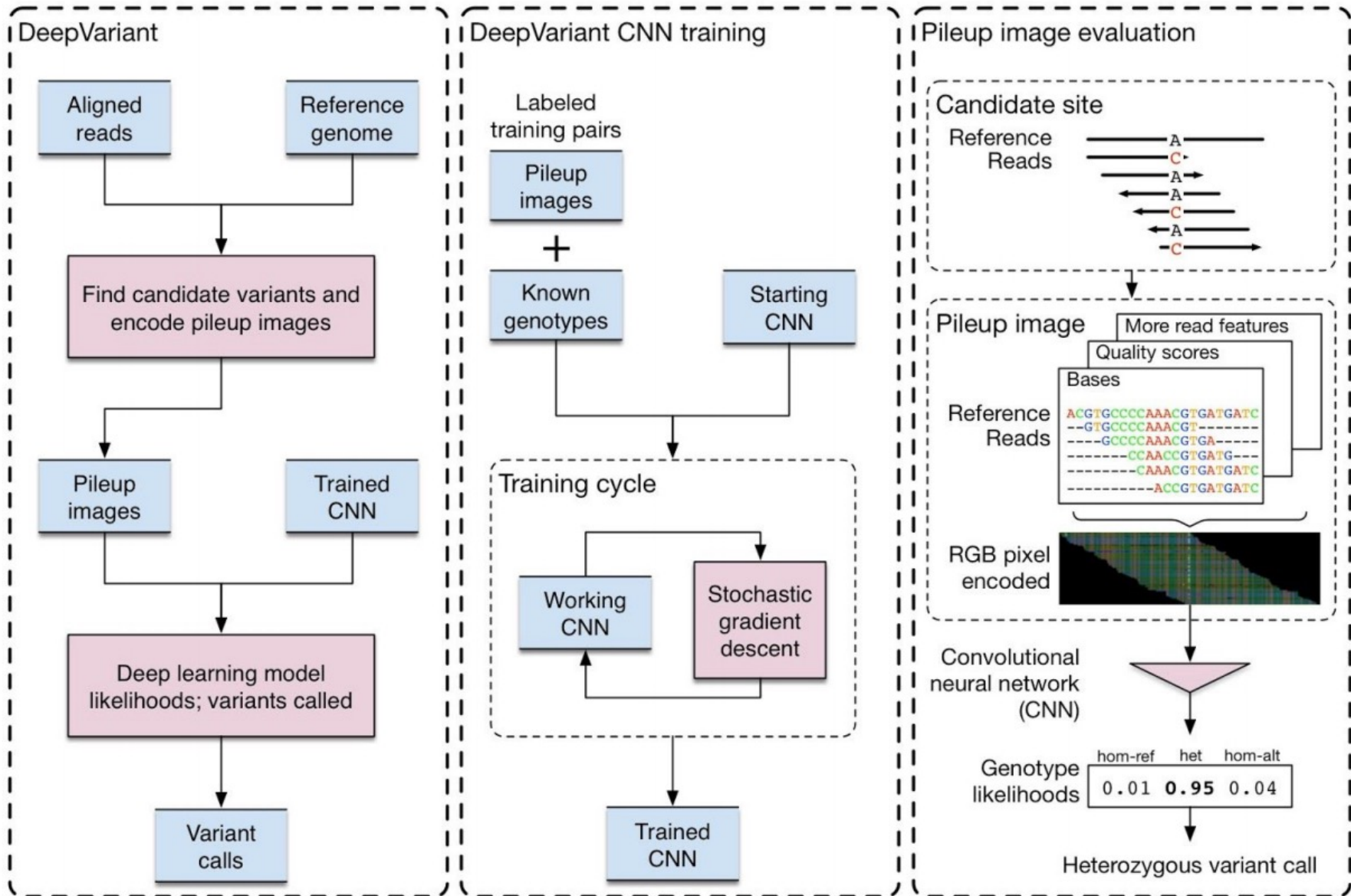
 © 1999 Nature America Inc. • <http://genetics.nature.com>

A general approach to single-nucleotide polymorphism discovery

Gabor T. Marth¹, Ian Korf¹, Mark D. Yandell¹, Raymond T. Yeh¹, Zhijie Gu², Hamideh Zakeri²,
Nathan O. Stitzel¹, LaDeana Hillier¹, Pui-Yan Kwok² & Warren R. Gish¹

This Bayesian statistical framework
has been adopted by most other
modern SNP/INDEL callers such as
FreeBayes, GATK, and samtools

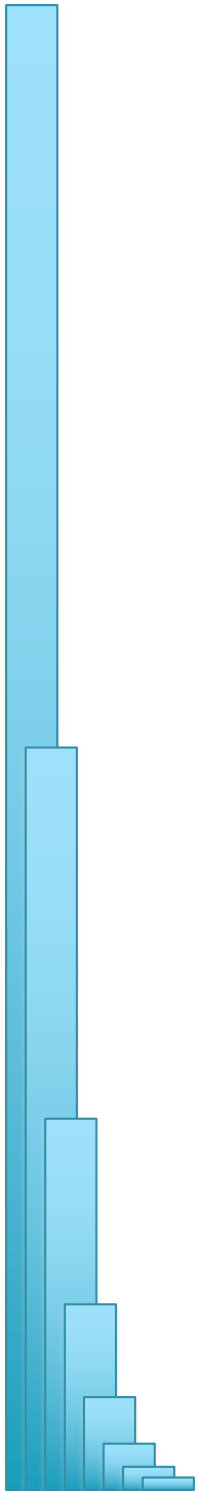
DeepVariant



Creating a universal SNP and small indel variant caller with deep neural networks

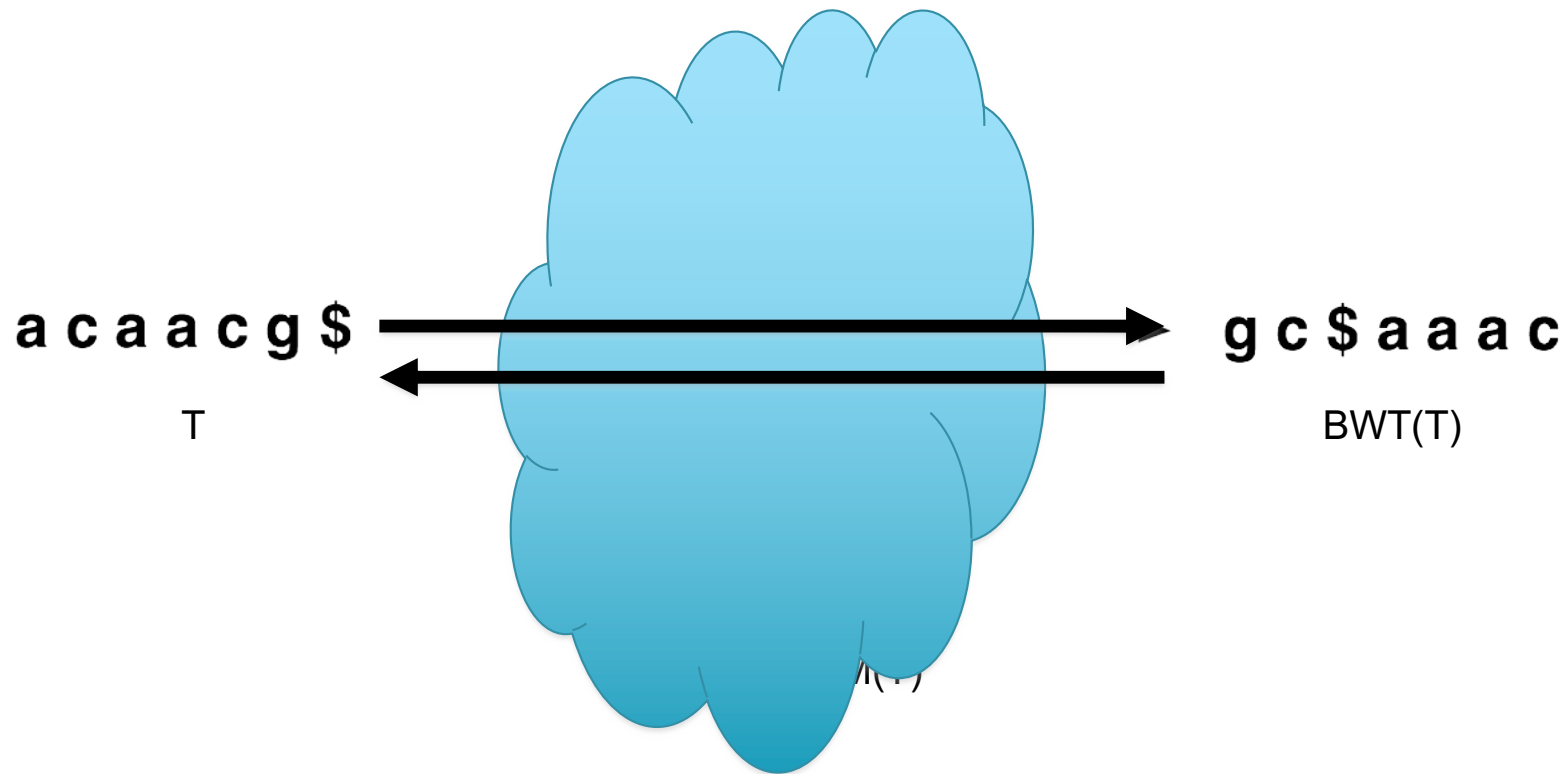
Poplin et al. (2018) Nature Biotechnology. <https://www.nature.com/articles/nbt.4235>

Bonus: BWT



Burrows-Wheeler Transform

- Reversible permutation of the characters in a text

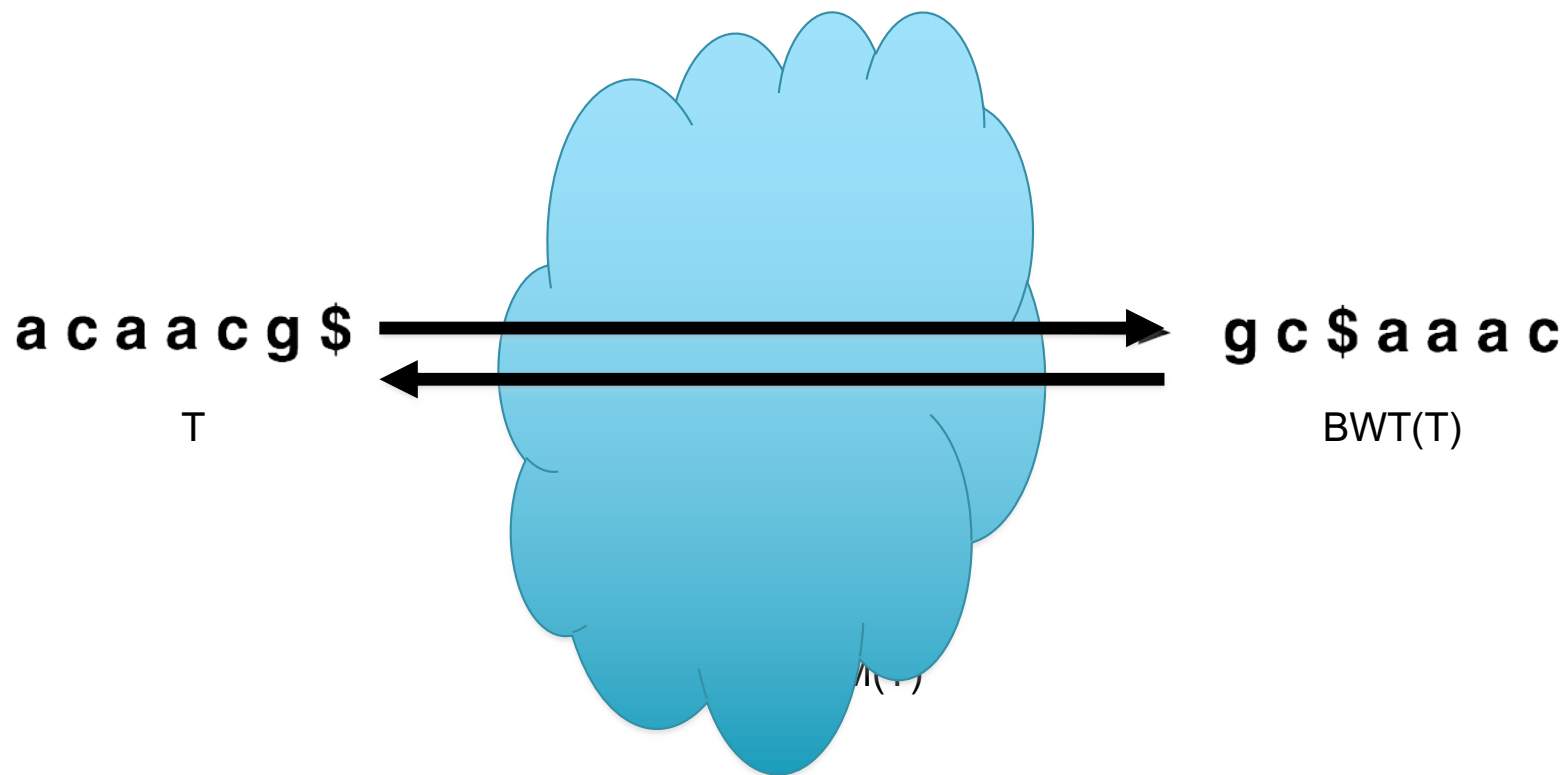


A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Reversible permutation of the characters in a text



A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text

a c a a c g \$ $\longrightarrow$

T

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text

a c a a c g \$ $\longrightarrow$ a c a a c g \$
T c a a c g \$ a
a a c g \$ a c
a c g \$ a c a
c g \$ a c a a
g \$ a c a a c
\$ a c a a c g

All cyclic permutations

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text

a c a a c g \$ $\longrightarrow$

T

\$	a	c	a	a	c	g
a	a	c	g	\$	a	c
a	c	a	a	c	g	\$
a	c	g	\$	a	c	a
c	a	a	c	g	\$	a
c	g	\$	a	c	a	a
g	\$	a	c	a	a	c

Sorted cyclic permutations
AKA Burrows Wheeler Matrix

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text

a c a a c g \$ $\longrightarrow$

T

\$	a	c	a	a	c	g
a	a	c	g	\$	a	c
a	c	a	a	c	g	\$
a	c	g	\$	a	c	a
c	a	a	c	g	\$	a
c	g	\$	a	c	a	a
g	\$	a	c	a	a	c

Sorted cyclic permutations
AKA Burrows Wheeler Matrix

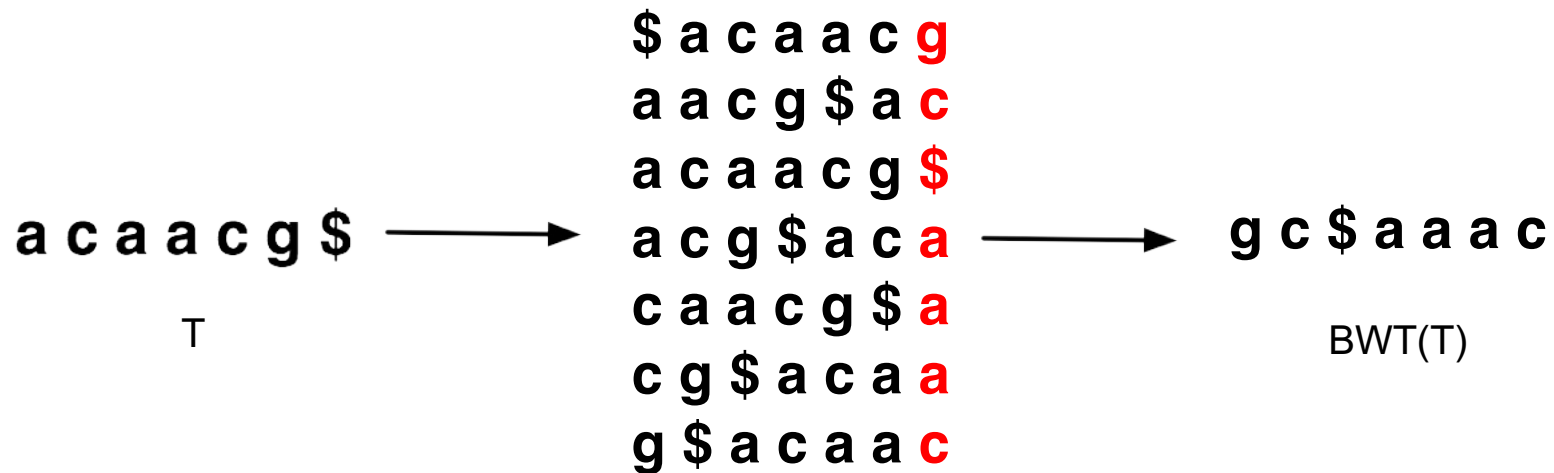
Last Column = Burrows Wheeler Transform

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Permutation of the characters in a text



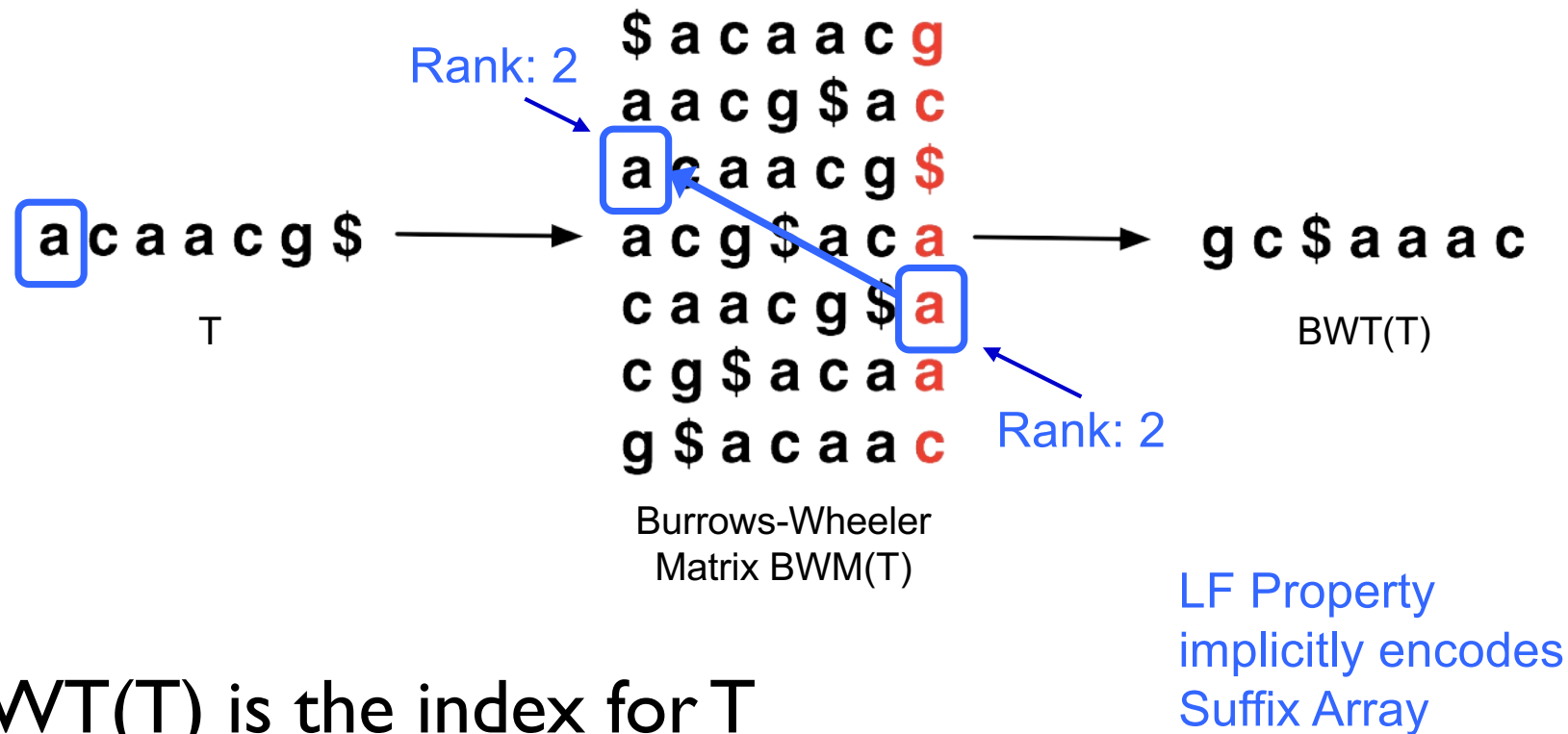
BWT(T) is the index for T

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

Burrows-Wheeler Transform

- Reversible permutation of the characters in a text



- BWT(T) is the index for T

A block sorting lossless data compression algorithm.

Burrows M, Wheeler DJ (1994) *Digital Equipment Corporation*. Technical Report 124

A TALE OF TWO CITIES

in three Books

BOOK THE FIRST. RECALLED TO LIFE

CHAPTER I
THE PERIOD

It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of Light, it was the season of Darkness, it was the spring of hope, it was the winter of despair, we had everything before us, we had nothing before us, we were all going direct to Heaven, we were all going direct the other way—in short, the period was so far the most extraordinary that ever was in the world.

A TALE OF TWO CITIES

In Three Books

BOOK THE FIRST. RECALLED TO LIFE

CHAPTER I

THE PERIOD

It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of Light, it was the season of Darkness, it was the spring of hope, it was the winter of despair, we had everything before us, we had nothing before us, we were all going direct to Heaven, we were all going direct the other way—in short, the period was so far the most extraordinary that ever existed, for good or for evil, in the superlatives of comparison only.

A TALE OF TWO CITIES

In Three Books

BOOK THE FIRST. RECALLED TO LIFE

CHAPTER I

THE PERIOD

It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of Light, it was the season of Darkness, it was the spring of hope, it was the winter of despair, we had everything before us, we had nothing before us, we were all going direct to Heaven, we were all going direct the other way—in short, the period was so far the most extraordinary that ever existed, for good or for evil, in the superlatives of comparison only.

A TALE OF TWO CITIES

In Three Books

BOOK THE FIRST. RECALLED TO LIFE

CHAPTER I

THE PERIOD

It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of Light, it was the season of Darkness, it was the spring of hope, it was the winter of despair, we had everything before us, we had nothing before us, we were all going direct to Heaven, we were all going direct the other way—in short, the period was so far the most extraordinary that ever existed, for good or for evil, in the superlatives of comparison only.

A TALE OF TWO CITIES

In Three Books

BOOK THE FIRST. RECALLED TO LIFE

CHAPTER I

THE PERIOD

It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of Light, it was the season of Darkness, it was the spring of hope, it was the winter of despair, we had everything before us, we had nothing before us, we were all going direct to Heaven, we were all going direct the other way—in short, the period was so far the most extraordinary that ever existed, for good or for evil, in the superlatives of comparison only.

A TALE OF TWO CITIES

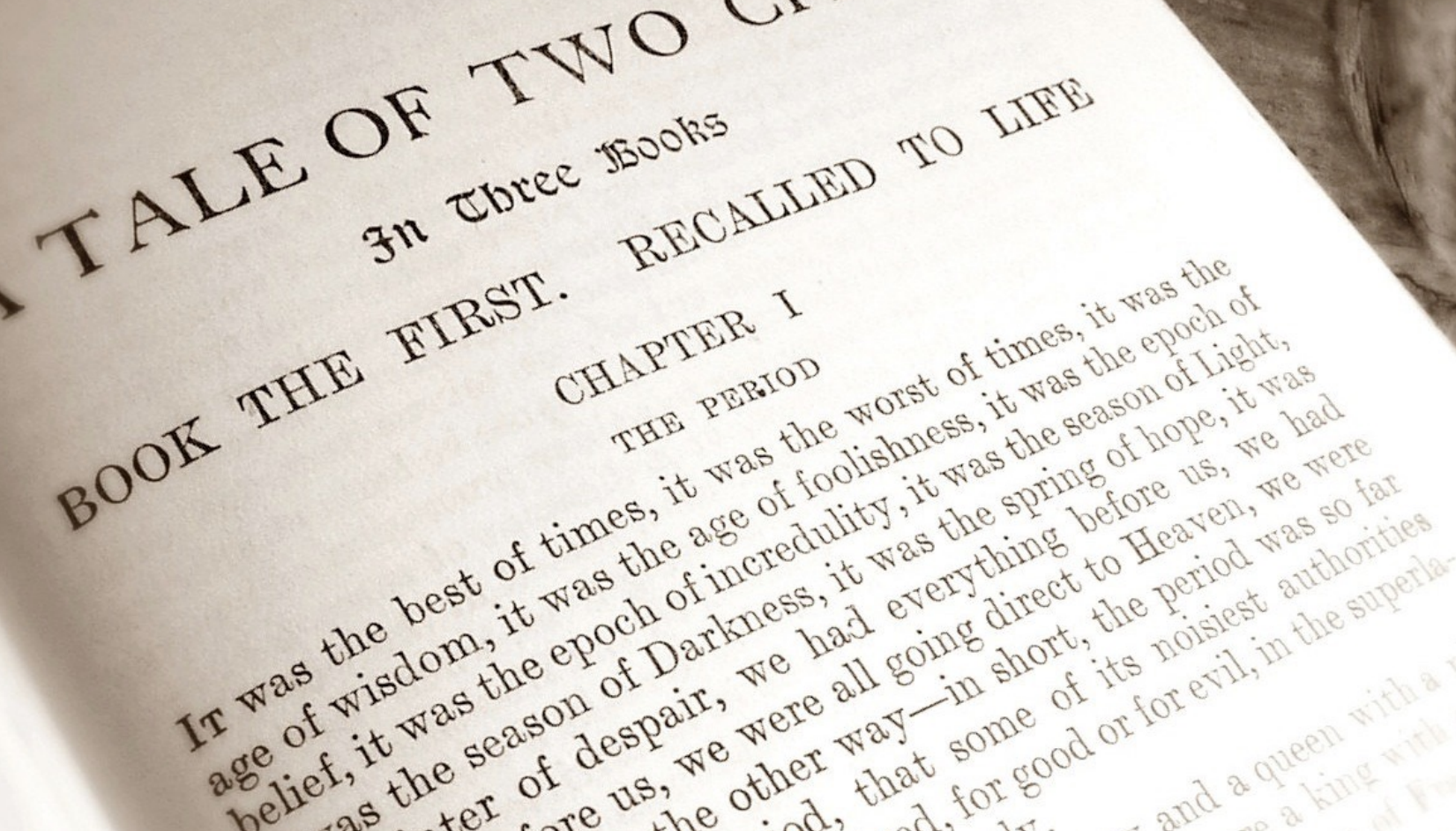
In Three Books

BOOK THE FIRST. RECALLED TO LIFE

CHAPTER I

THE PERIOD

It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of Light, it was the season of Darkness, it was the spring of hope, it was the winter of despair, we had everything before us, we had nothing before us, we were all going direct to Heaven, we were all going direct the other way—in short, the period was so far the most extraordinary that ever existed, for good or for evil, in the superlatives of comparison only.



A TALE OF TWO CITIES

In Three Books

BOOK THE FIRST. RECALLED TO LIFE

CHAPTER I

THE PERIOD

It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness, it was the epoch of belief, it was the epoch of incredulity, it was the season of Light, it was the season of Darkness, it was the spring of hope, it was the winter of despair, we had everything before us, we had nothing before us, we were all going direct to Heaven, we were all going direct the other way—in short, the period was so far the most extraordinary that ever existed, for good or for evil, in the superlatives of comparison only.

Run Length Encoding

ref[614] :

It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa_s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint_er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_that_some_of_its_noisiest_authorities_insisted_on_its_being_received,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.\$

Run Length Encoding:

- Replace a “run” of a character X with a single X followed by the length of the run
- GAAAAAAAAATTACA => GA8T2ACA (reverse is also easy to implement)
- If your text contains numbers, then you will need to use a (slightly) more sophisticated encoding

Run Length Encoding

ref[614]:

It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa_s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint_er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_that_some_of_its_noisiest_authorities_insisted_on_its_being_received,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.\$

rle(ref) [614]:

It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_of_wisdom,_it_was_the_age_of_fo2lishnes2,_it_was_the_epoch_of_belief,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa_s_the_season_of_Darknes2,_it_was_the_spring_of_hope,_it_was_the_wint_er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us,_we_were_al2_going_direct_to_Heaven,_we_were_al2_going_direct_the_o ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_that_some_of_its_noisiest_authorities_insisted_on_its_being_received,_for_go2d_or_for_evil,_in_the_superlative_degre2_of_comparison_only.\$

Run Length Encoding

ref[614] :

It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa_s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint_er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_that_some_of_its_noisiest_authorities_insisted_on_its_being_received,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.\$

bwt[614] :

.dlmssfty sesdtrsns_y__\$_yfofeeeeetggsfefefggedrofr,llreef-,fs,,,,,,,,, ,nfrsdnnherghettedndeteegenstee,sssst,esssnssffteedtttttttttttr,, ,eeefehh__p__fpDwwwwwwwwweehl_ew_____eoo_neeeoaaeoo_____sephhrrhvh hwwegmghhhhhhhkrrwvhhssHrrrvtrribbdbcbs__thwpppvmmirdnnib__eoooooo oooooo_____eennnnnnnai_____ecc__ttttttttttttttttttts_tsgltsLlvtt__hhoor e_wrraddwlors_____r__lteirillre_ouaanooiioeooooiihkiiiiiiio__iei tsppioi_____ggnodsc_sss_gfhf_fffhwh_nsmo__uee_sioooaeeeeoo_ii cgppeeaoaeooeesseuutetaaaaaaaaaaaaaai__ei_in__aaie_eeerei_hrsssnacciIi iiiiiisn_____oyoui__a_iids__aiaae_____tlar

Run Length Encoding

ref[614] :

It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa_s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint_er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_that_some_of_its_noisiest_authorities_insisted_on_its_being_received,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.\$

bwt[614] :

.dlmssftysestdtrsns_y__\$yfofeeeeetggsfefefggeedrofr,llreef-,fs,,,,,,,,, ,nfrsdnnhereghettedndeteegenstee,sssst,esssnssffteedtttttttttttr,, ,eeefehh__p__fpDwwwwwwwwweehl_ew_____eoo_neeeoaaeoo_____sephhrrhvh hwwegmghhhhhhhkrrrwhhssHrrrvtrribbdbcbvs__thwwpppvmmirdnnib__eoooooo oooooo_____eennnnnnnai_____ecc__ttttttttttttttttttts_tsgltsLlvtt__hhoor e_wrraddwlors_____r__lteirillre_ouaanooiioeooooiihkiiiiiiio__iei tsppioi_____ggnodsc_sss_gfhf_fffhwh_nsmo__uee_sioooaeeeeoo_ii cgppeeaoaeooeesseuutetaaaaaaaaaaaaaai__ei_in__aaie_eeerei_hrsssnacciIi iiiiiisn_____tlar

Why does the BWT tend to make runs in english text?

Run Length Encoding

bwt[614] :

```
.dlmssftysestrns_y__$yfofeeeeetggsfefefggedrofr,llreef-,fs,,,,,,,,,  
,,nfrsdnnherghettedndeteegenstee,sssst,esssnssffteedtttttttttttr,,  
,,eeefehh__p__fpDwwwwwwwwweehl_ew_____eoo_neeeoaaeeo____sephhrrhvh  
hwwegmghhhhhhhkrrwwhhssHrrrvtrribbdbcbvs__thwwpppvmmirdnnib__eoooooo  
oooooo_____eennnnnnnai__ecc_tttttttttttttttttts_tsgltsLlvt__hhoor  
e_wrraddwlors_____r_lteirillre_ouaanooiioeooooiihkiiiiio__iei  
tsppioi_____ggnodsc_sss_gfhf_fffhwh_nsmo__uee_sioooaeeeeoo_ii  
cgppeeaoaeooesseuutetaaaaaaaaaaaaaai__ei_in__aaie_eeerei_hrsssnacciIi  
iiiiisn_____oyoui__a_iids__aiaee_____tlar
```

rle(bwt) [464] :

```
.dlms2ftysestrns_y_2$yfofe4tg2sfefefg2e2drofr,l2re2f-,fs,9nfrsdn2  
herghet2edndete2ge2nste2,s5t,es3ns2f2te2dt10r,4e3feh2_2p_2fpDw11e2h  
l_ew_5eo2_ne3oa2eo2_4seph2r2hvh2w2egmgh7kr2w2h2s2Hr3vtr2ib2dbcbvs_2t  
hw2p3vm2irdn2ib_2eo12_4e2n6a2i_3ec2_2t18s_tsgltsLlvt2_3h2o2re_wr2ad2  
wlors_9r_2lteiril2re_oua2no2i2oeo4i3hki6o_2ieitsp2ioi_12g2nodsc_s3_g  
fhf_f3hwh_nsmo_2ue2_sio3ae4o2_i2cgp2e2aoaeo2e2s2eu2teta11i_2ei_in_2a  
2ie_e3rei_hrs3nac2i2Ii7sn_15oyoui_2a_i3ds_2ai2ae2_21tlar
```

Run Length Encoding

bwt[614] :

```
.dlmssftysestrns_y__$yfofeeeeetggsfefefggedrofr,llreef-,fs,,,,,,,,,
,,nfrsdnnherghettedndeteegenstee,sssst,esssnssfsteedttrtr,,
,,eeefehh__p__fpDwwwwwwweehl_ew_____eoo_neeeoaaeoo_____sephhrrhvh
hwwegmghhhhhhhkrrwwhhssHrrrvtrribbdbcbvs__thwwpppvmmirdnnib__eooooo
ooooo_____eennnnnnaai__ecc__ttttttttttttttts_tsgltsLlvt__hhoor
e_wrraddwlors_____r__lteirillre_ouaanooioeooooiihkiiiiio__iei
tsppioi_____ggnodsc_sss_gfhf_fffhwh_nsmo__uee_sioooaeeeeoo__ii
cgppeaoaeooesseuutetaaaaaaaaai__ei_in__aaie_eeerei_hrssnacciIi
iiiiisn_____oyoui__a_iids__aiaee_____tlar
```

rle(bwt) [464] :

```
.dlms2ftysestrns_y_2$yfofe4tg2sfefefg2e2drofr,l2re2f-,fs,9nfrsdn2
herghet2edndete2ge2nste2,s5t,es3ns2f2te2dt10r,4e3feh2_2p_2fpDw11e2h
l_ew_5eo2_ne3oa2eo2_4seph2r2hvh2w2egmgh7kr2w2h2s2Hr3vtr2ib2dbcbvs_2t
hw2p3vm2irdn2ib_2eo12_4e2n6a2i_3ec2_2t18s_tsgltsLlvt2_3h2o2re_wr2ad2
wlors_9r_2lteiril2re_oua2no2i2oeo4i3hki6o_2ieitsp2ioi_12g2nodsc_s3_g
fhf_f3hwh_nsmo_2ue2_sio3ae4o2_i2cgp2e2aoaeo2e2s2eu2tet11i_2ei_in_2a
2ie_e3rei_hrs3nac2i2Ii7sn_15oyoui_2a_i3ds_2ai2ae2_21tlar
```

Run Length Encoding

ref[614] :

It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa_s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint_er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_that_some_of_its_noisiest_authorities_insisted_on_its_being_received,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.\$

rle(bwt) [464] :

.dlms2ftysesdtrsns_y_2\$_yfofe4tg2sfefefg2e2drofr,l2re2f-,fs,9nfrsdn2 hereghet2edndete2ge2nste2,s5t,es3ns2f2te2dt10r,4e3feh2_2p_2fpDw11e2h l_ew_5eo2_ne3oa2eo2_4seph2r2hvh2w2egmgh7kr2w2h2s2Hr3vtr2ib2dbcbvs_2t hw2p3vm2irdn2ib_2eo12_4e2n6a2i_3ec2_2t18s_tsgltsLlvt2_3h2o2re_wr2ad2 wlors_9r_2lteiril2re_oua2no2i2oeo4i3hki6o_2ieitsp2ioi_12g2nodsc_s3_g fhf_f3hwh_nsmo_2ue2_sio3ae4o2_i2cgp2e2aoaao2e2s2eu2tet11i_2ei_in_2a 2ie_e3rei_hrs3nac2i2Ii7sn_15oyoui_2a_i3ds_2ai2ae2_21tlar

Run Length Encoding

ref[614] :

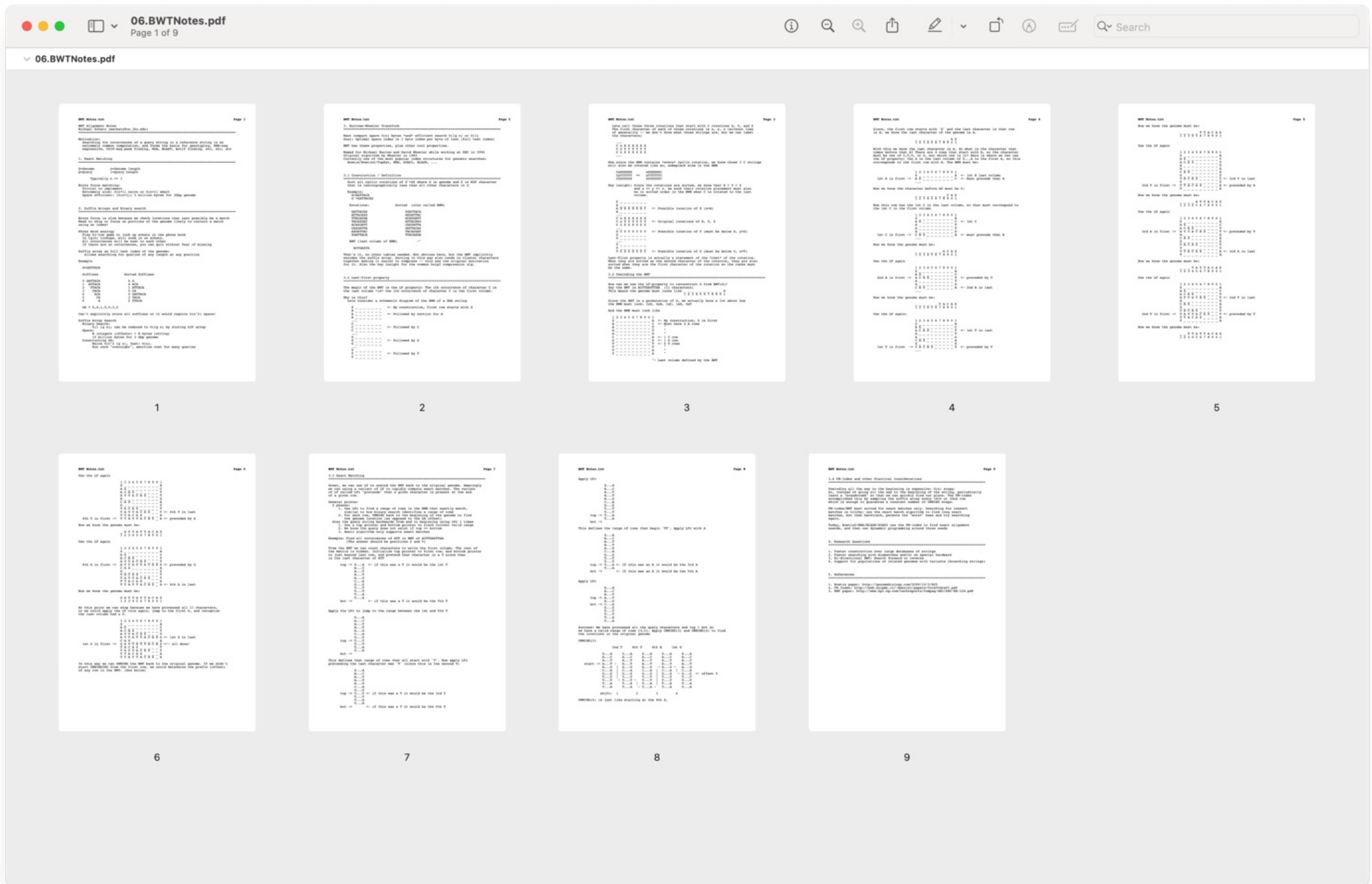
```
It_was_the_best_of_times,_it_was_the_worst_of_times,_it_was_the_age_
of_wisdom,_it_was_the_age_of_foolishness,_it_was_the_epoch_of_belief
,_it_was_the_epoch_of_incredulity,_it_was_the_season_of_Light,_it_wa
s_the_season_of_Darkness,_it_was_the_spring_of_hope,_it_was_the_wint
er_of_despair,_we_had_everything_before_us,_we_had_nothing_before_us
,_we_were_all_going_direct_to_Heaven,_we_were_all_going_direct_the_o
ther_way_-_in_short,_the_period_was_so_far_like_the_present_period,_
that_some_of_its_noisiest_authorities_insisted_on_its_being_received
,_for_good_or_for_evil,_in_the_superlative_degree_of_comparison_only.$
```

rle(bwt) [464] :

```
.dlms2ftysesdtrsns_y_2$_yfofe4tg2sfefefg2e2drofr,l2re2f-,fs,9nfrsdn2
hereghet2edndete2ge2nste2,s5t,es3ns2f2te2dt10r,4e3feh2_2p_2fpDw11e2h
l_ew_5eo2_ne3oa2eo2_4seph2r2hvh2w2egmgh7kr2w2h2s2Hr3vtr2ib2dbcbvs_2t
hw2p3vm2irdn2ib_2eo12_4e2n6a2i_3ec2_2t18s_tsgltsLlvt2_3h2o2re_wr2ad2
wlors_9r_2lteiril2re_oua2no2i2oeo4i3hki6o_2ieitsp2ioi_12g2nodsc_s3_g
fhf_f3hwh_nsmo_2ue2_sio3ae4o2_i2cgp2e2aoaao2e2s2eu2tet11i_2ei_in_2a
2ie_e3rei
```

Saved 614-464 = 150 bytes (24%) with zero loss of information!

Common to save 50% to 90% on real world files with bzip2



Recompute the original text when the BWT is ACTTGA\$TTAA

Burrows-Wheeler Transform

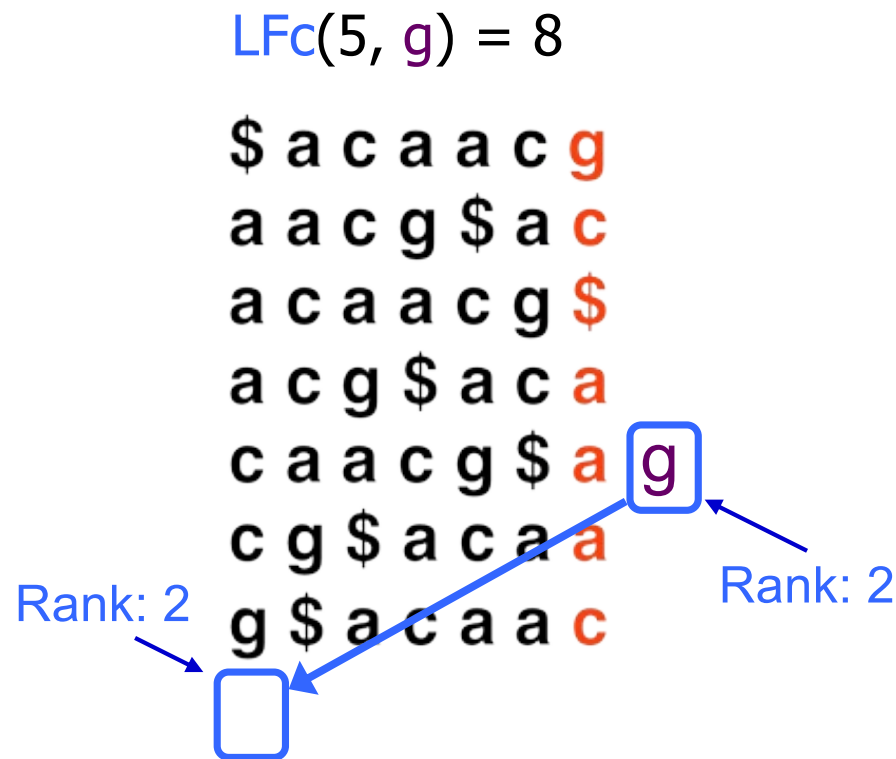
- Recreating T from BWT(T)
 - Start in the first row and apply **LF** repeatedly, accumulating predecessors along the way



[Decode this BWT string: ACTGA\$TTA]

BWT Exact Matching

- **LFc**(r, c) does the same thing as **LF**(r) but it ignores r's actual final character and “pretends” it's c:

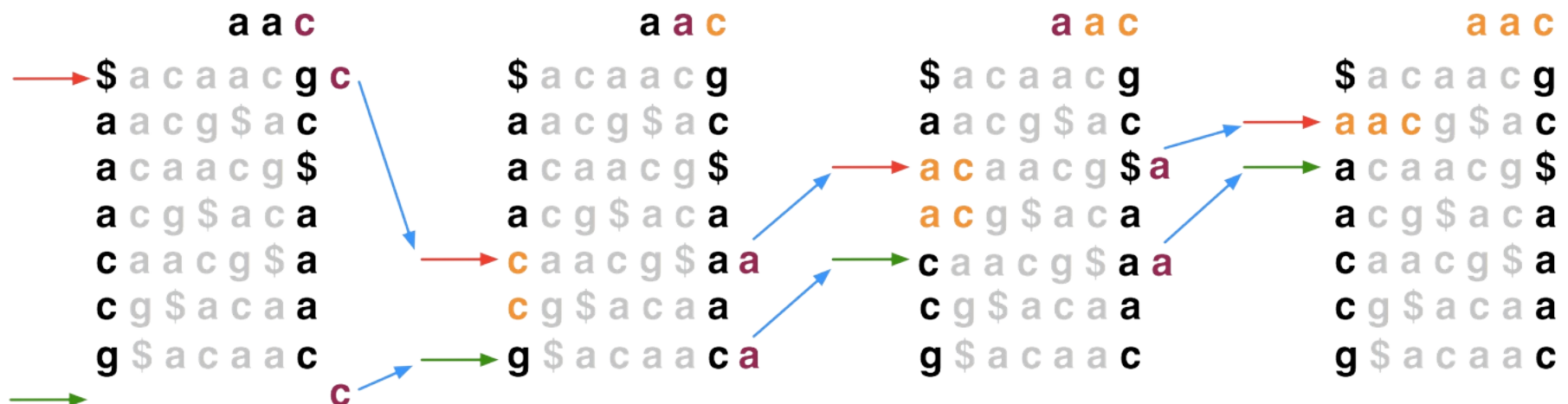


BWT Exact Matching

- Start with a range, (**top**, **bot**) encompassing all rows and repeatedly apply **LFc**:

top = **LFc**(**top**, **qc**); **bot** = **LFc**(**bot**, **qc**)

qc = the next character to the left in the query



Ferragina P, Manzini G: Opportunistic data structures with applications. *FOCS. IEEE Computer Society; 2000.*

[Search for TTA this BWT string: ACTGA\$TTA]